



Project: Visual Question Answering

Dương Đình Thắng

Dương Trường Bình

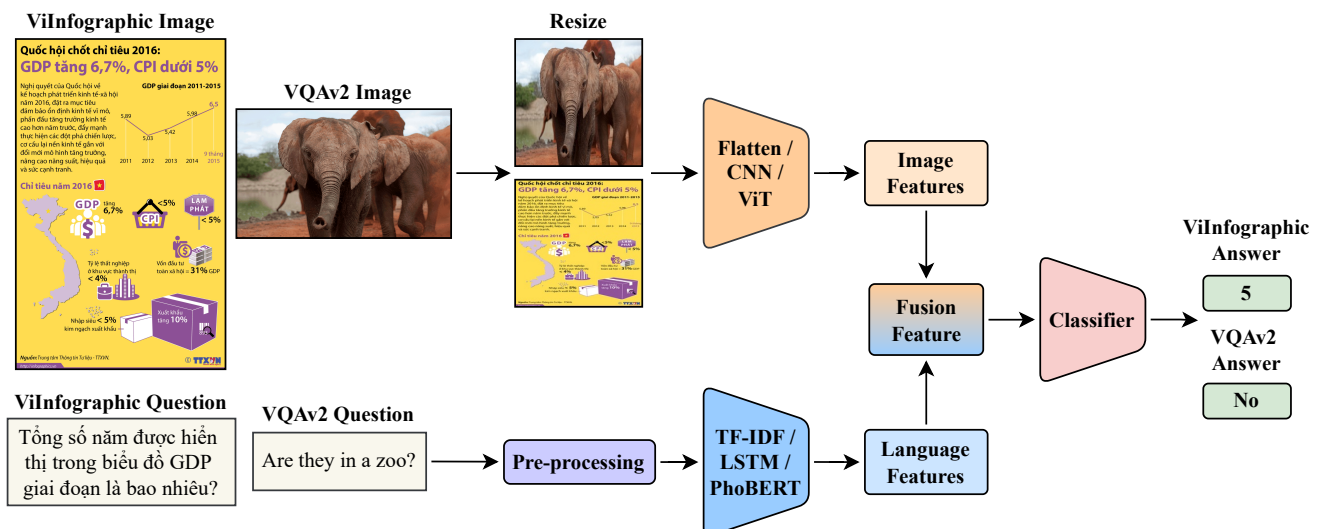
Nguyễn Anh Khôi

Đình Quang Vinh

I. Dẫn nhập

Visual question answering (VQA) là một bài toán nằm tại điểm giao thoa giữa hai lĩnh vực lớn là computer vision (CV) và natural language processing (NLP). Đây được cho là một trong những bài toán đầy thách thức khi yêu cầu hệ thống phải thực hiện xử lý đồng thời cả hai khả năng về CV và NLP để hiểu dữ liệu đầu vào và đưa ra dự đoán chính xác nhất.

Trong bối cảnh bùng nổ nguồn dữ liệu đa dạng từ văn bản, hình ảnh đến video như hiện nay, VQA đóng vai trò then chốt trong việc xây dựng các hệ thống trí tuệ nhân tạo thông minh hơn. Các hệ thống này hứa hẹn khả năng tương tác tự nhiên với con người thông qua cả hình ảnh và ngôn ngữ, mở ra nhiều ứng dụng thực tiễn trong đời sống.



Hình 1: Hình minh họa tổng quan toàn bộ dự án xây dựng mô hình VQA trên bộ dữ liệu VQA2 [1] và ViInfographic [2].

⁰Để tiện theo dõi, các bạn có thể tham khảo bảng các thuật ngữ được dùng trong bài tại Mục VII..

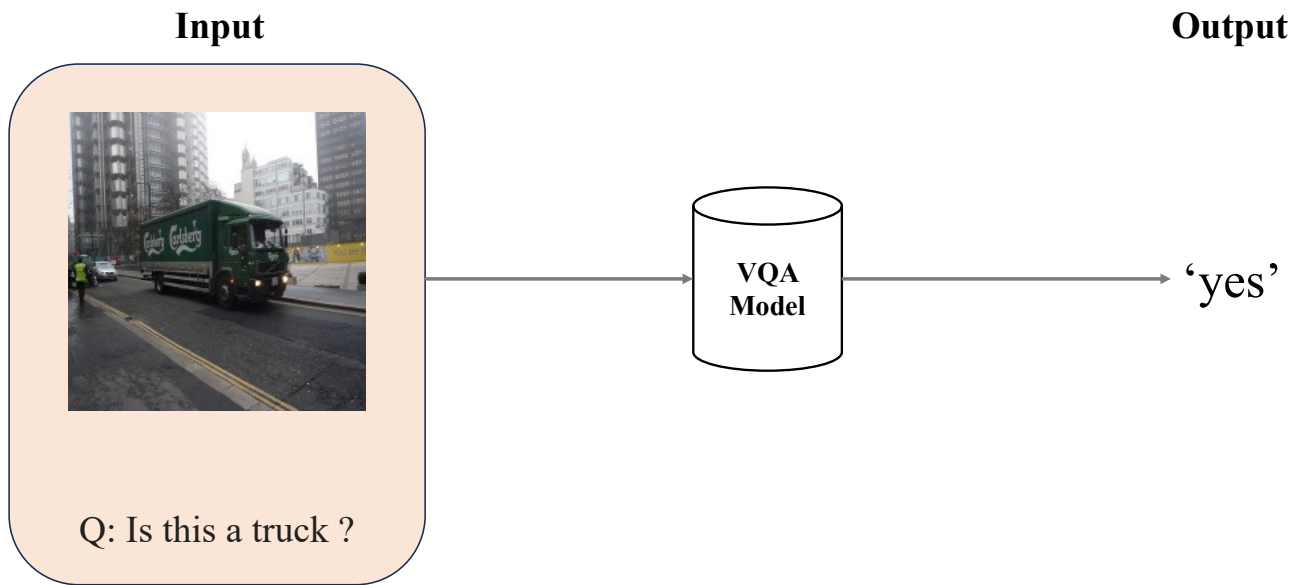
Mục lục

I.	Dẫn nhập	1
I.1.	Mục tiêu	3
I.2.	Input và output	3
I.3.	Nguyên lý hoạt động chung	4
I.4.	Các hướng phát triển hiện nay	5
II.	Nội dung project	9
II.1.	Các kiến trúc thực nghiệm	9
II.2.	Dataset sử dụng	11
II.3.	Quy trình cài đặt code tổng quan	14
III.	Cài đặt chương trình	16
III.1.	Kiến trúc 1: Multi-layer Perceptron (MLP)	16
III.2.	Kiến trúc 2: CNN+LSTM	26
III.3.	Kiến trúc 3: ViT+BERT	30
IV.	Thực nghiệm	34
IV.1.	Thiết lập thực nghiệm	34
IV.2.	Độ đo đánh giá	34
IV.3.	Kết quả chính	37
IV.4.	Thực nghiệm về các cách kết hợp đặc trưng	39
IV.5.	Thực nghiệm về thiết kế classifier	41
V.	Các hướng mở rộng	43
VI.	Câu hỏi trắc nghiệm	49
VII.	Tài liệu tham khảo	57
	Bảng thuật ngữ	59
	Phụ lục	61

I.1. Mục tiêu

Mục tiêu cốt lõi của bài toán VQA là xây dựng một hệ thống trí tuệ nhân tạo có khả năng đưa ra câu trả lời chính xác cho một câu hỏi liên quan đến nội dung của một bức ảnh đầu vào. Để đạt được điều này, mô hình cần phải đáp ứng được ba yêu cầu cơ bản sau đây.

1. **Image understanding:** hệ thống cần trích xuất được các đặc trưng, nhận diện các đối tượng, các thuộc tính liên quan và hiểu được mối quan hệ không gian giữa các đối tượng trong ảnh.
2. **Language understanding:** mô hình phải phân tích được cú pháp và ngữ nghĩa của câu hỏi để xác định chính xác ý định mà người hỏi muốn hướng tới.
3. **Multimodal fusion:** hệ thống cần kết hợp thông tin thu được từ cả hình ảnh và văn bản để thực hiện quá trình suy luận và đưa ra câu trả lời hợp lý nhất.



Hình 2: Ví dụ minh họa một Input/Output của bài toán VQA.

I.2. Input và output

Theo minh họa ở Hình 2, bài toán VQA có thể được định nghĩa với đầu vào và đầu ra như sau:

- **Input:** bao gồm một hình ảnh I và một câu hỏi Q liên quan đến hình ảnh đó.
- **Output:** bao gồm một câu trả lời dự đoán $\hat{a}$.

Thông thường, VQA được mô hình hóa dưới dạng bài toán classification trên một không gian các câu trả lời cố định. Cách tiếp cận này giúp đơn giản hóa quá trình đánh giá và tối ưu hóa việc huấn luyện so với việc sinh văn bản tự do. Cụ thể, mô hình sẽ lựa chọn đáp án từ một tập

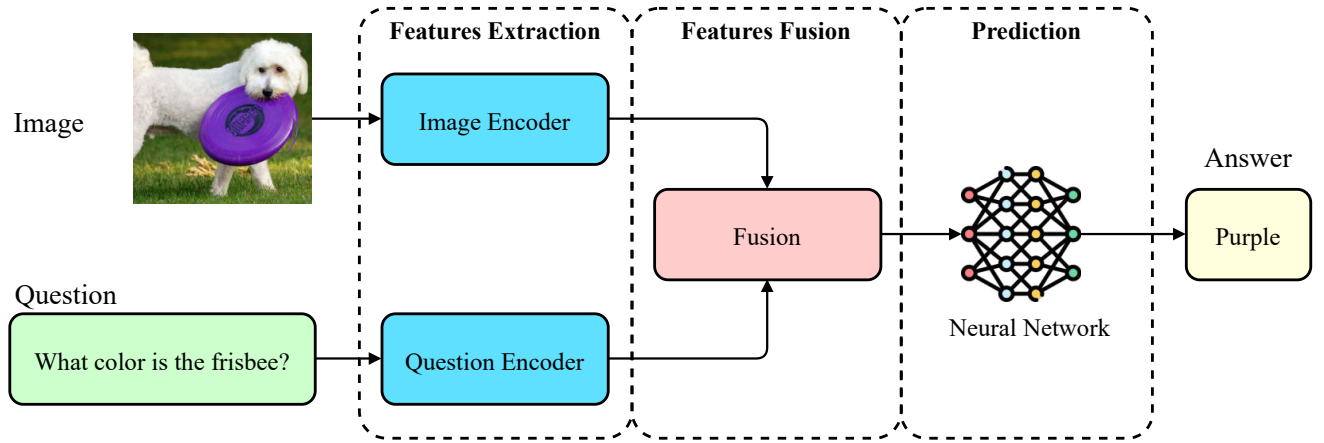
$\mathcal{A}$ chứa K câu trả lời phổ biến nhất xuất hiện trong dữ liệu huấn luyện. Bài toán đi tìm câu trả lời tối ưu $\hat{a}$ được biểu diễn thông qua công thức:

$$\hat{a} = \arg \max_{a \in \mathcal{A}} P(a | I, Q; \theta) \quad (1)$$

Trong đó:

- $\hat{a}$: là câu trả lời đầu ra được mô hình dự đoán.
- $\mathcal{A} = \{a_1, a_2, \dots, a_K\}$: là không gian các câu trả lời khả dĩ.
- a : là một câu trả lời ứng viên bất kỳ thuộc tập $\mathcal{A}$.
- θ : là tập hợp các tham số của mô hình cần được huấn luyện.
- $P(a | I, Q; \theta)$: là xác suất có điều kiện để câu trả lời là a , khi biết trước hình ảnh I và câu hỏi Q , được tính toán bởi mô hình với tham số θ .

I.3. Nguyên lý hoạt động chung



Hình 3: Minh họa kiến trúc của một mô hình VQA cơ bản. Input là một mẫu dữ liệu được lấy từ bộ VQAv2 [1].

Trước khi đi sâu vào chi tiết từng kiến trúc mô hình cụ thể, chúng ta cần nắm bắt nguyên lý hoạt động nền tảng của các hệ thống VQA. Hầu hết các mô hình VQA kinh điển, đặc biệt là các mô hình baseline được sử dụng để đánh giá trên bộ dữ liệu VQAv2 [1], đều tuân theo một quy trình xử lý chuẩn gồm các bước chính như Hình 3:

- **Image feature extraction:** hình ảnh đầu vào được đưa qua một bộ trích xuất để chuyển đổi từ ảnh sang một vector đặc trưng chứa các thông tin thị giác quan trọng của bức ảnh.
- **Text feature extraction:** tương tự như ảnh, câu hỏi được mã hóa thành vector số học để mô hình có thể xử lý và hiểu được ngữ nghĩa.

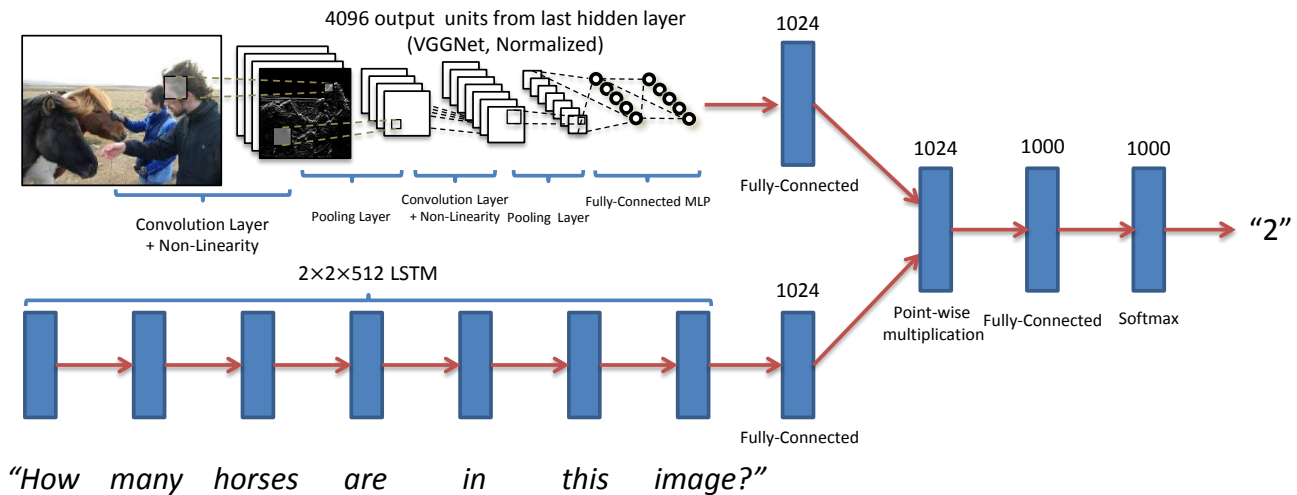
- **Fusion:** đây là bước quan trọng nhất nhằm liên kết hai luồng thông tin. Các đặc trưng từ ảnh và văn bản được kết hợp lại với nhau để tạo thành một vector biểu diễn chung cho cả hai nhánh.
- **Prediction:** vector tổng hợp cuối cùng được đưa vào một classifier để tính toán xác suất phân bố trên không gian câu trả lời và chọn ra câu trả lời chính xác nhất.

I.4. Các hướng phát triển hiện nay

Bài toán VQA đã trải qua nhiều giai đoạn phát triển mạnh mẽ, từ các mô hình học máy đơn giản đến các mô hình ngôn ngữ lớn đa phương thức phức tạp. Dựa trên các tài liệu tổng quan và các công trình tiêu biểu, chúng ta có thể phân loại các hướng tiếp cận chính như sau:

I.4.1. Multimodal fusion truyền thống

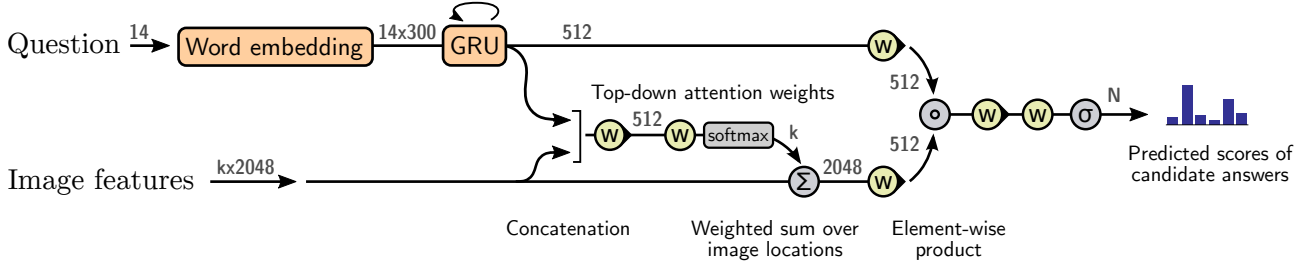
Đây là hướng tiếp cận khởi đầu và cơ bản nhất của bài toán VQA, tiêu biểu là hai công trình [1, 3]. Ý tưởng chủ đạo của nhóm phương pháp này là ánh xạ cả hình ảnh và câu hỏi vào một không gian đặc trưng chung.



Hình 4: Minh họa kiến trúc multimodal fusion kết hợp đặc trưng. Ảnh trích từ [3].

Cụ thể, quá trình xử lý được minh họa chi tiết trong Hình 4. Tại nhánh hình ảnh, các mạng nơ-ron tích chập như VGGNet hoặc ResNet thường được sử dụng để trích xuất đặc trưng thị giác. Song song đó, tại nhánh ngôn ngữ, câu hỏi được mã hóa thành vector thông qua các mạng nơ-ron hồi quy RNN hoặc LSTM. Cuối cùng, hai vector đặc trưng này được kết hợp bằng các phép toán đơn giản như phép cộng, phép nhân hoặc phép nối để tạo ra đầu vào cho bộ phân lớp. Mặc dù đơn giản và hiệu quả về mặt tính toán, các phương pháp này thường gặp khó khăn trong việc nắm bắt các mối quan hệ phức tạp giữa các vùng cụ thể trong ảnh với các từ khóa trong câu hỏi.

I.4.2. Cơ chế attention

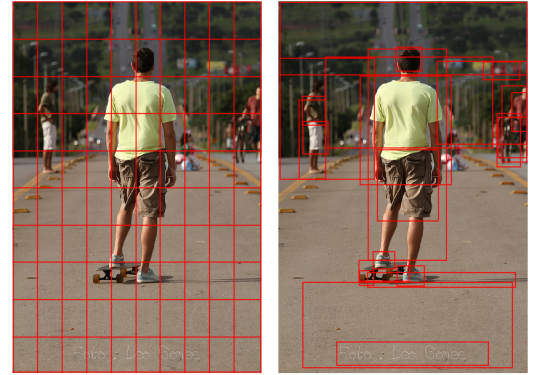


Hình 5: Kiến trúc của mô hình BUTD kết hợp đặc trưng đối tượng từ Faster R-CNN và cơ chế attention định hướng bởi câu hỏi. Ảnh trích từ [4].

Để khắc phục hạn chế của việc nén toàn bộ bức ảnh vào một vector duy nhất, cơ chế attention đã được áp dụng rộng rãi. Thay vì xử lý toàn bộ hình ảnh như nhau, mô hình học cách tập trung vào các vùng ảnh quan trọng có liên quan trực tiếp đến câu hỏi.

Nổi bật trong hướng đi này là kiến trúc bottom-up and top-down (UpDn/BUTD) [4]. Như minh họa ở Hình 5, mô hình kết hợp đặc trưng câu hỏi với các đặc trưng ảnh để dự đoán câu trả lời, trong đó trọng số chú ý (*top-down attention*) được suy ra từ câu hỏi nhằm nhấn mạnh các vùng thông tin liên quan.

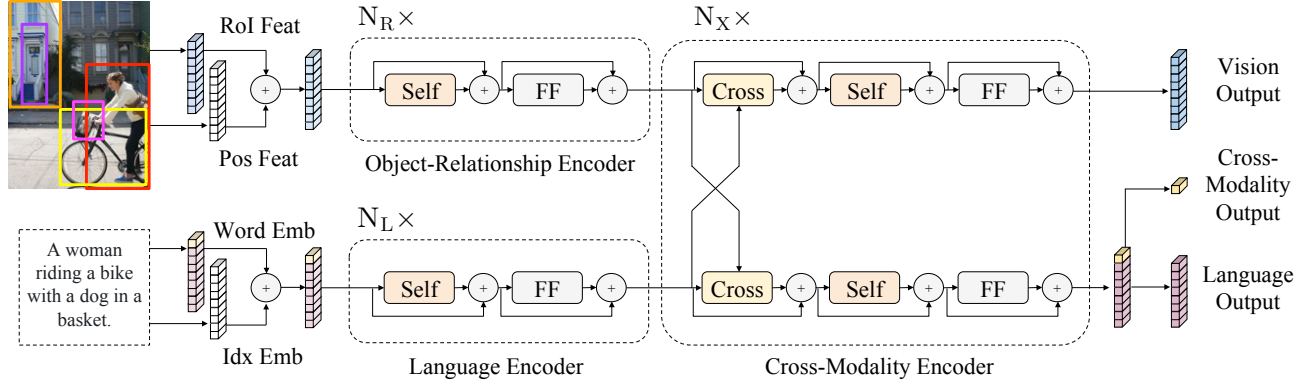
Điểm đột phá của BUTD nằm ở cơ chế *bottom-up*: thay vì chia ảnh theo lưới cố định, mô hình sử dụng Faster R-CNN để trích xuất đặc trưng trên từng vùng đối tượng cụ thể. So sánh trực quan giữa hai cách trích xuất theo lưới và theo vùng đối tượng được thể hiện ở Hình 6. Sự kết hợp giữa *bottom-up* (đề xuất vùng đối tượng) và *top-down* (gán trọng số chú ý theo câu hỏi) đã cải thiện đáng kể hiệu năng của các hệ thống VQA.



Hình 6: So sánh cách trích xuất đặc trưng theo lưới cố định (trái) và theo các vùng đối tượng (phải). Ảnh trích từ [4].

I.4.3. Transformer và pre-trained multimodal models

Sự phát triển của kiến trúc Transformer trong xử lý ngôn ngữ tự nhiên đã lan tỏa mạnh mẽ sang lĩnh vực thị giác máy tính. Các mô hình như ViLBERT [5] hay LXMERT [6] đều dựa trên nền tảng này để học mối quan hệ giữa hình ảnh và văn bản thông qua quá trình pre-training trên lượng dữ liệu lớn.

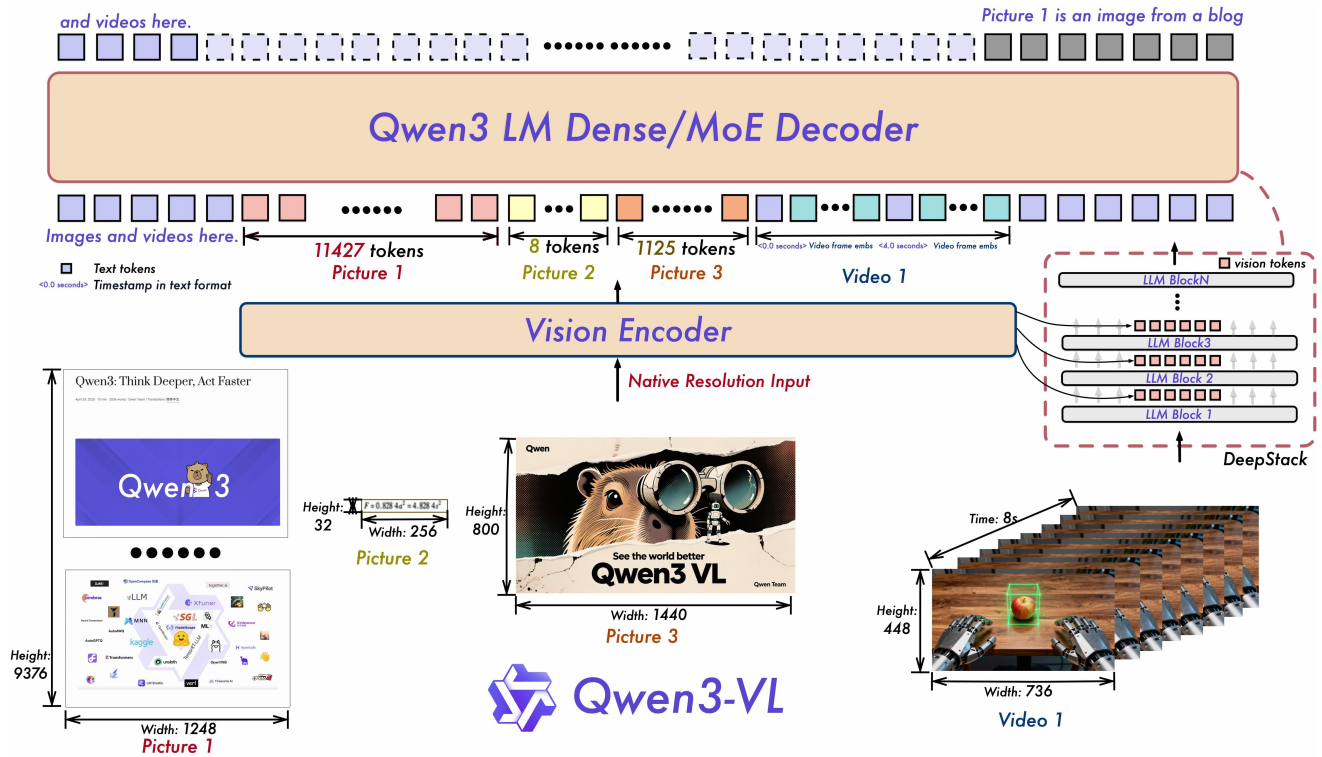


Hình 7: Kiến trúc mạng LXMERT với cơ chế mã hóa chéo cross-modality encoder giúp kết hợp đặc trưng hình ảnh và ngôn ngữ. Ảnh trích từ [6].

Điển hình là kiến trúc LXMERT được mô tả chi tiết tại Hình 7. Như trên sơ đồ, mô hình thiết kế hai nhánh mã hóa riêng biệt bao gồm object-relationship encoder cho hình ảnh và language encoder cho văn bản. Sau đó, thông tin từ hai luồng dữ liệu này được kết hợp thông qua khối cross-modality encoder. Tại đây, các lớp attention chéo giúp mô hình học được sự tương tác sâu sắc giữa các vùng ảnh và từ ngữ.

I.4.4. Large vision-language models

Không dừng lại ở các kiến trúc mã hóa chuyên biệt, xu hướng nghiên cứu hiện nay đã chuyển dịch mạnh mẽ sang các mô hình ngôn ngữ thị giác lớn hay còn gọi là large vision-language models. Bước tiến này đánh dấu sự chuyển đổi từ việc chỉ hiểu hình ảnh sang khả năng sinh văn bản và lập luận linh hoạt.



Hình 8: Kiến trúc tổng quát của dòng mô hình Qwen3-VL, đại diện cho xu hướng tích hợp bộ mã hóa thị giác vào LLM để xử lý đa phương thức. Ảnh trích từ [7].

Tiêu biểu cho làn sóng này là sự ra đời của BLIP [8] với cơ chế căn chỉnh thống nhất giữa hình ảnh và văn bản. Tiếp đó, LLaVA [9] đã tạo nên bước ngoặt lớn khi tích hợp sức mạnh của các mô hình ngôn ngữ lớn để nâng cao khả năng tuân theo yêu cầu phức tạp của người dùng. Đặc biệt trong giai đoạn hiện nay, các đại diện tiên tiến nhất như Qwen-VL [7] hay InternVL [10] đang liên tục thiết lập các chuẩn mực mới về hiệu năng. Như minh họa tại Hình 8, các kiến trúc này cho phép tích hợp bộ mã hóa thị giác tiên tiến vào mô hình ngôn ngữ lớn, từ đó xử lý được hình ảnh độ phân giải cao và thực hiện các tác vụ suy luận đa bước phức tạp.

II. Nội dung project

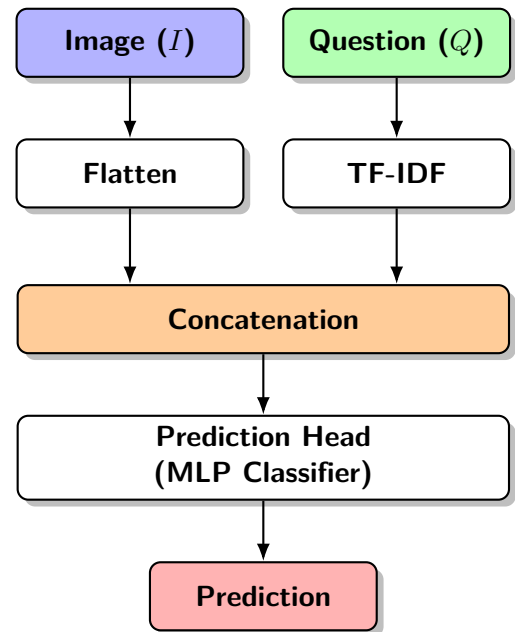
Dựa trên khung sườn lý thuyết đã thảo luận tại mục **I.3.**, dự án sẽ triển khai cài đặt và so sánh thực nghiệm ba phương pháp tiếp cận bài toán VQA. Các kiến trúc này được lựa chọn theo trình tự từ đơn giản đến phức tạp, với sự thay đổi linh hoạt trong kỹ thuật trích xuất đặc trưng và cơ chế kết hợp đa phương thức. Qua đó, dự án không chỉ đánh giá hiệu quả của từng mô hình mà còn minh họa rõ nét sự phát triển trong bài toán này.

II.1. Các kiến trúc thực nghiệm

Kiến trúc 1: Multi-Layer Perceptron

Phương pháp này đóng vai trò là mô hình cơ sở để so sánh với các kiến trúc phức tạp hơn. Quy trình xử lý dữ liệu được thiết kế dựa trên các kỹ thuật trích xuất đặc trưng truyền thống kết hợp với mạng multi-layer perceptron (MLP).

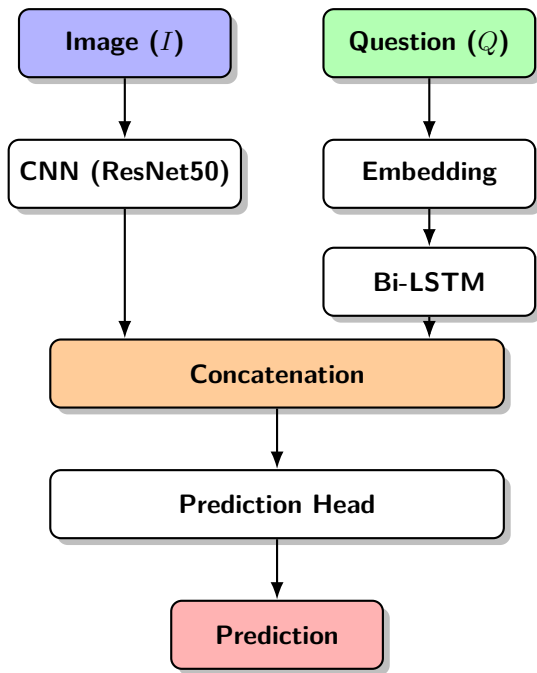
- **Image processing:** hình ảnh đầu vào dưới dạng các tensor đa chiều sẽ trải qua quá trình làm phẳng hay còn gọi là flattening. Thao tác này biến đổi ma trận điểm ảnh thành một vector đặc trưng một chiều để phù hợp với đầu vào của mạng MLP.
- **Text processing:** câu hỏi đầu vào được mã hóa thành vector số học thông qua kỹ thuật term frequency-inverse document frequency (TF-IDF). Phương pháp này giúp biểu diễn tầm quan trọng của các từ khóa trong câu hỏi dựa trên tần suất xuất hiện của chúng trong toàn bộ tập dữ liệu huấn luyện.
- **Fusion and classification:** hệ thống thực hiện kết hợp hai luồng thông tin bằng phép nối vector. Vector đặc trưng tổng hợp sau đó được đưa vào khối Prediction Head. Tại đây, mạng sử dụng các lớp tuyến tính kết hợp với hàm kích hoạt phi tuyến và kỹ thuật dropout để trích xuất đặc trưng và dự đoán xác suất của câu trả lời cuối cùng.



Hình 9: Kiến trúc MLP với TF-IDF.

Kiến trúc 2: CNN kết hợp LSTM

Phương pháp này tận dụng sức mạnh của các mô hình học sâu chuyên biệt để xử lý đặc trưng của từng loại dữ liệu trước khi kết hợp chúng.



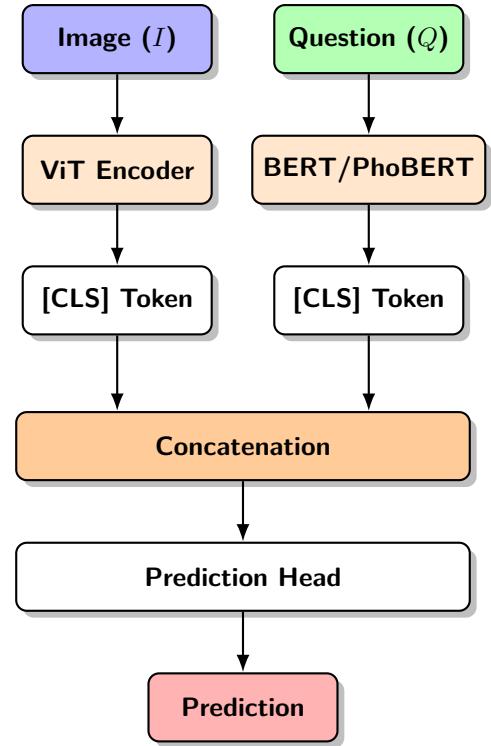
Hình 10: Kiến trúc CNN + Bi-LSTM.

- **Image branch:** sử dụng convolutional neural network (CNN) làm backbone trích xuất đặc trưng. Cụ thể, mô hình ResNet50 đã được huấn luyện trước trên tập dữ liệu ImageNet được dùng để chuyển đổi ảnh đầu vào thành vector đặc trưng. Các tham số của mạng được đóng băng khi huấn luyện để giữ lại các đặc trưng tổng quát.
- **Text branch:** câu hỏi được xử lý thông qua lớp embedding để chuyển từ vệtng sang không gian vector. Sau đó, chuỗi vector này được đưa qua mạng long short-term memory hai chiều (Bi-LSTM). Vector đặc trưng cuối cùng được tạo bằng cách nối các hidden state từ hai hướng duyệt xuôi và ngược.
- **Fusion and classification:** vector đặc trưng từ ảnh và văn bản được kết hợp thông qua phép nối và đi qua khối Prediction Head. Khối này bao gồm các lớp tuyến tính, hàm kích hoạt ReLU và dropout để thực hiện trích xuất đặc trưng đa phương thức và dự đoán câu trả lời cuối cùng.

Kiến trúc 3: ViT kết hợp BERT

Phương pháp này áp dụng hoàn toàn kiến trúc Transformer để xử lý cả hai luồng dữ liệu, cho phép mô hình học các biểu diễn ngữ nghĩa phong phú hơn nhờ cơ chế self-attention.

- **Visual encoder:** sử dụng mô hình Vision Transformer (ViT) phiên bản Base. Ảnh đầu vào được chia nhỏ và xử lý như một chuỗi token. Mô hình trích xuất vector đại diện toàn cục cho ảnh thông qua token phân loại [CLS].
- **Text encoder:** sử dụng mô hình ngôn ngữ bidirectional encoder representations from Transformers (BERT), hoặc PhoBERT đối với dữ liệu tiếng Việt, để mã hóa câu hỏi. Câu hỏi được tokenize và đưa qua các lớp Transformer encoder. Tương tự như nhánh ảnh, vector đặc trưng ngữ nghĩa của câu hỏi được lấy từ token [CLS] tại lớp ẩn cuối cùng.
- **Fusion and classification:** hai vector đặc trưng [CLS] từ nhánh thị giác và ngôn ngữ được kết hợp bằng phép nối. Vector tổng hợp đi vào khối Prediction Head đóng vai trò là bộ phân loại cuối cùng. Cả hai encoder ViT và BERT đều được đóng băng tham số khi huấn luyện để tận dụng tri thức đã học trước đó.



Hình 11: Kiến trúc ViT + BERT.

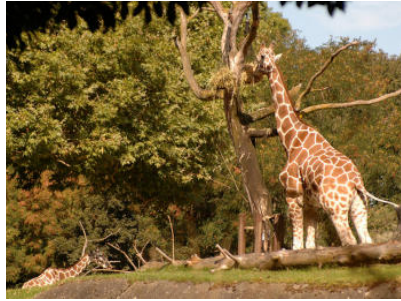
II.2. Dataset sử dụng

Để đánh giá hiệu quả của các phương pháp trên, dự án sẽ tiến hành thực nghiệm trên hai bộ dữ liệu đặc trưng, đại diện cho hai ngôn ngữ và hai loại hình ảnh khác nhau:

II.2.1. VQAv2 - Visual Question Answering version 2.0



Câu hỏi: What color are the dishes?
 Câu trả lời: pink and yellow



Câu hỏi: Are the animals eating?
 Câu trả lời: yes



Câu hỏi: Are all the flowers white?
 Câu trả lời: no



Câu hỏi: What is the bear called?
 Câu trả lời: teddy bear



Câu hỏi: Which way do the arrows point?
 Câu trả lời: left

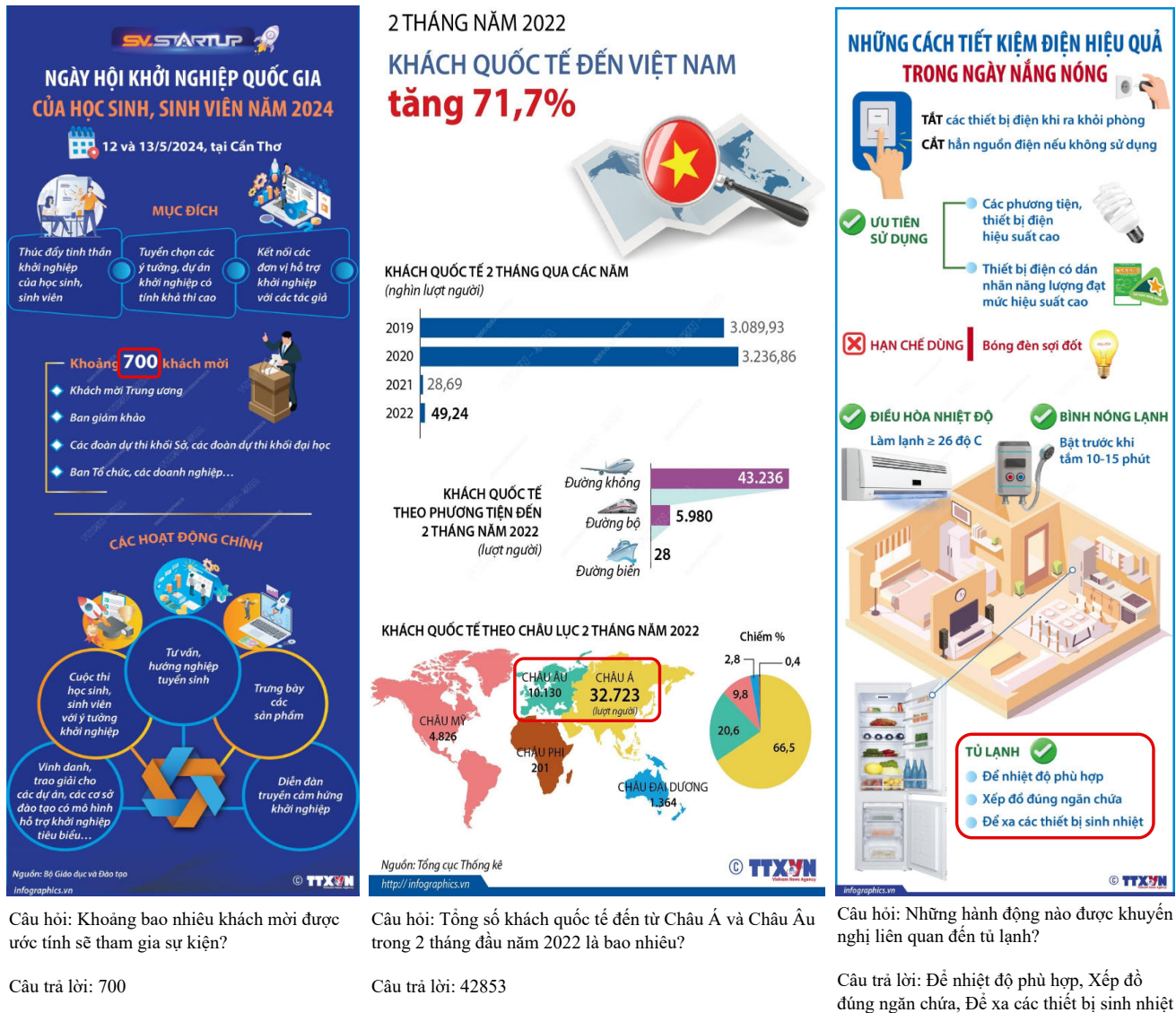


Câu hỏi: What are the animals doing?
 Câu trả lời: grazing

Hình 12: Minh hoạt một số mẫu dữ liệu trong bộ VQAv2 [1].

- **Tổng quan:** VQAv2 [1], mở rộng từ bộ VQAv1 [3], là một trong những bộ benchmark được coi là tiêu chuẩn cho bài toán VQA. Bộ dữ liệu được xây dựng dựa trên hình ảnh thực tế từ bộ dữ liệu MS COCO. Các mẫu dữ liệu được minh họa trong Hình 12.
- **Đặc điểm nổi bật:** điểm cải tiến lớn nhất của VQAv2 so với phiên bản trước là cơ chế *balanced pairs*. Với cùng một câu hỏi, tập dữ liệu cung cấp hai hình ảnh khác nhau dẫn đến hai câu trả lời trái ngược, ví dụ như “Yes” và “No”. Cơ chế này buộc mô hình phải thực sự hiểu nội dung ảnh thay vì chỉ đoán mò dựa trên xác suất, thiên vị về mặt ngôn ngữ, điều này còn gọi là language bias.
- **Thách thức:** đòi hỏi mô hình phải có khả năng nhận diện đối tượng phong phú và hiểu các mối quan hệ không gian phức tạp trong môi trường tự nhiên.

II.2.2. ViInfographicVQA - Vietnamese Infographic VQA



Hình 13: Minh họa một số mẫu dữ liệu trong bộ ViInfographicVQA [2]. Các mẫu dữ liệu ViInfographicVQA đòi hỏi khả năng đọc hiểu biểu đồ và văn bản cao hơn.

- **Tổng quan:** ViInfographicVQA [2] là một bộ dữ liệu VQA tiếng Việt, tập trung vào khía cạnh hỏi đáp trên các hình ảnh infographic, poster, và tài liệu - những loại hình có mật độ chữ xuất hiện trên hình cao. Một vài mẫu dữ liệu được trình bày trong Hình 13.
- **Đặc điểm nổi bật:** khác với ảnh tự nhiên, thông tin trong infographic được trình bày dưới dạng cấu trúc như bảng biểu, biểu đồ, tiêu đề. Câu hỏi thường yêu cầu mô hình phải thực hiện suy luận đa bước dựa trên nội dung văn bản trong ảnh.
- **Thách thức:** đây là bài toán khó hơn VQA truyền thống vì yêu cầu tích hợp thêm khả năng nhận dạng văn bản và xử lý ngôn ngữ tiếng Việt, đặc biệt là các từ chuyên ngành hoặc số liệu thống kê.

Thống kê về hai bộ dữ liệu

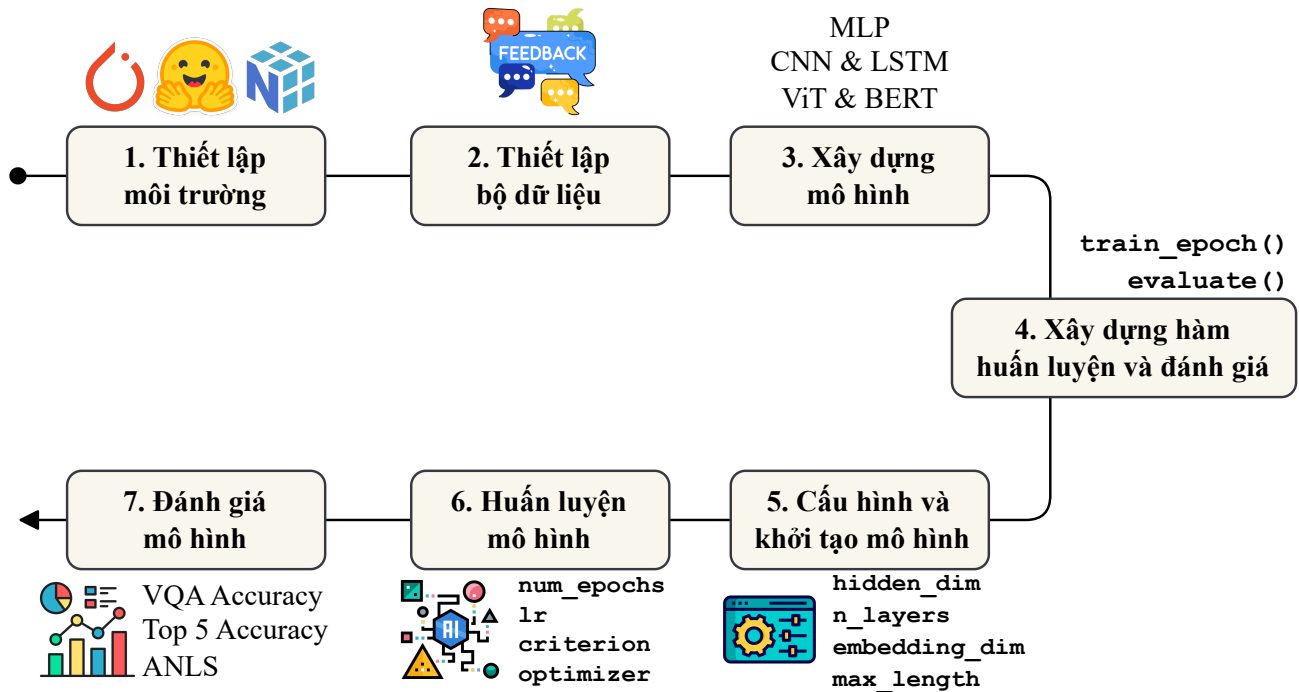
Bảng 1: Thống kê số lượng mẫu trong hai bộ dữ liệu VQAv2 và ViInfographicVQA.

Dataset	Language	Original Size		Project Subset	
		Images	QA Pairs	Images	QA Pairs
VQAv2	English	~204,000	~1,100,000	4,417	7,000
ViInfographicVQA	Vietnamese	6,747	20,409	1,570	7,000

Để đảm bảo tính khả thi cho việc triển khai và thực nghiệm mô hình trong phạm vi dự án, một tập con được trích xuất từ hai bộ dữ liệu gốc. Quy mô cụ thể của tập dữ liệu được chọn lọc so với kích thước ban đầu được thống kê chi tiết trong Bảng 1.

II.3. Quy trình cài đặt code tổng quan

Về tổng quan, toàn bộ các phần cài đặt code trong project này đều sẽ đi qua các bước sau:



Hình 14: Tổng quan các giai đoạn cài đặt dự án.

Cụ thể, theo minh họa từ Hình 14, ta có:

- 1. Thiết lập môi trường**: import các thư viện cần thiết và cố định seed để đảm bảo khả năng tái tạo lại kết quả cho mỗi lần chạy (reproducibility), đồng thời cấu hình thiết bị có trong máy để huấn luyện mô hình (CPU/GPU).

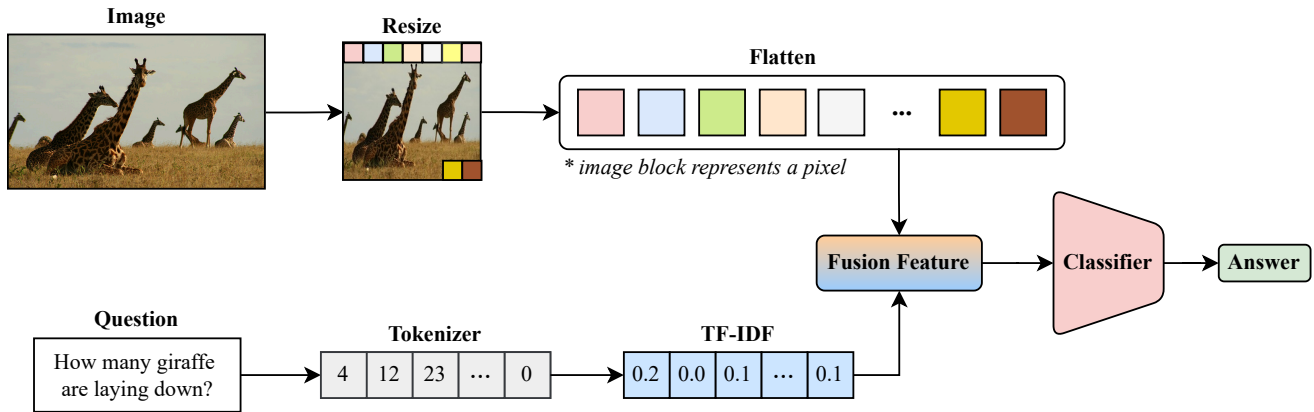
2. **Thiết lập bộ dữ liệu:** tải/đọc dữ liệu, xây dựng `Dataset` và `DataLoader`, định nghĩa các phép tiền xử lý ảnh.
3. **Xây dựng mô hình:** triển khai các kiến trúc theo hướng fusion bao gồm image encoder, text encoder, cơ chế fusion và prediction head.
4. **Xây dựng hàm huấn luyện và đánh giá:** cài đặt vòng lặp `train_epoch()` và `evaluate()` để tính loss và các độ đo đánh giá trên val/test.
5. **Cấu hình và khởi tạo mô hình:** thiết lập các siêu tham số quan trọng (ví dụ `hidden_dim`, số tầng, `max_len`) và khởi tạo mô hình cùng các thành phần huấn luyện.
6. **Huấn luyện mô hình:** cấu hình `num_epochs`, learning rate, loss function và optimizer; tiến hành huấn luyện theo epoch, theo dõi chỉ số trên validation và lưu checkpoint tốt nhất.
7. **Đánh giá mô hình:** chạy đánh giá trên tập test và tổng hợp kết quả cuối cùng để phục vụ so sánh giữa các thiết lập/kiến trúc.

III. Cài đặt chương trình

Trong phần này, dựa vào nội dung project đã được mô tả ở phần II., ta tiến hành triển khai cài đặt code cho từng hàng mục, bao gồm ba kiến trúc (1) Multi-layer Perceptron (MLP), (2) CNN+LSTM và (3) ViT+BERT. Toàn bộ các code được cài đặt trên Google Colab.

III.1. Kiến trúc 1: Multi-layer Perceptron (MLP)

Chúng ta triển khai hướng tiếp cận MLP (baseline) cho bài toán VQA theo pipeline đơn giản: trích xuất đặc trưng ảnh và câu hỏi, kết hợp hai đặc trưng bằng phép nối, sau đó dự đoán câu trả lời bằng một khối MLP.



Hình 15: Hình minh họa chi tiết mô hình VQA với MLP.

Cụ thể, theo minh họa ở Hình 15, kiến trúc được cấu thành bởi:

- **Image encoder:** không dùng backbone học sâu. Ảnh sau tiền xử lý được *flatten* trực tiếp thành vector đặc trưng kích thước $3 \times H \times W$ (H là chiều cao còn W là chiều rộng của hình).
- **Text encoder:** mã hoá câu hỏi bằng *TF-IDF* với số chiều cố định (trong code: `tfidf_dim=1000`).
- **Fusion:** *concat* hai vector đặc trưng ảnh và text.
- **Prediction head:** một MLP gồm (1) bottleneck layer (Linear-ReLU-Dropout) và (2) classifier (Linear-ReLU-Dropout-Linear) để dự đoán phân phối trên tập nhãn câu trả lời.

Bắt đầu chương trình, đoạn code sau khởi tạo các thư viện cần thiết cho quá trình tiền xử lý dữ liệu, xây dựng mô hình, huấn luyện và đánh giá:

1. Thiết lập môi trường

```

1 import json
2 import torch
3 import torch.nn as nn
4 from torch.utils.data import Dataset, DataLoader
5 from PIL import Image
6 import os
7 from sklearn.feature_extraction.text import TfidfVectorizer
8 import torchvision.transforms as transforms
9 import random
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from collections import Counter

```

Để đảm bảo kết quả có thể tái lập, ta cố định seed cho các thư viện ngẫu nhiên và thiết lập thiết bị chạy (GPU nếu có):

```

1 def set_seed(seed=42):
2     random.seed(seed)
3     np.random.seed(seed)
4     torch.manual_seed(seed)
5     torch.cuda.manual_seed_all(seed)
6     torch.backends.cudnn.deterministic = True
7     torch.backends.cudnn.benchmark = False
8
9 set_seed(42)
10 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
11 print(f"Using device: {device}")

```

Đoạn code sau tải bộ dữ liệu đã được đóng gói sẵn và giải nén vào thư mục trên máy (link tải dataset được đính kèm ở phần VII.):

2. Thiết lập bộ dữ liệu

```

1 !wget 1wQoF9F7916oVtAJPBMEr7Kiu7ySPyzDc
2 !unzip -q data.zip

```

Ta xây dựng lớp `VQADataset` để đọc annotation từ file JSON, tải ảnh tương ứng, áp dụng biến đổi tiền xử lý và ánh xạ câu trả lời về chỉ số trong từ điển nhãn:

```

1 class VQADataset(Dataset):
2     def __init__(self, json_path, data_dir, transform=None, vocab=None):
3         with open(json_path, "r", encoding="utf-8") as f:
4             self.data = json.load(f)
5
6         self.data_dir = data_dir
7         self.transform = transform

```

```

8         self.vocab = vocab
9
10        def __len__(self):
11            return len(self.data)
12
13        def __getitem__(self, idx):
14            item = self.data[idx]
15
16            # Load image
17            img_path = os.path.join(self.data_dir, item["image_path"])
18            image = Image.open(img_path).convert("RGB")
19            if self.transform:
20                image = self.transform(image)
21
22            # Question
23            question = item["question"]
24
25            # Answer
26            answer = item["answer"]
27            answer_idx = self.vocab.get(answer, -1)
28
29            # Multiple answers for VQA Accuracy
30            answers = item.get("answers", [answer])
31
32            return {
33                "image": image,
34                "question": question,
35                "answer": answer_idx,
36                "raw_answer": answer,
37                "answers": answers
38            }

```

Đoạn code sau định nghĩa pipeline tiền xử lý ảnh (resize, chuẩn hoá) cho train/validation theo cùng kích thước đầu vào:

```

1  def get_transforms(img_size=224):
2      train_transform = transforms.Compose([
3          transforms.Resize((img_size, img_size)),
4          transforms.ToTensor(),
5          transforms.Normalize(mean=[0.485, 0.456, 0.406],
6                                std=[0.229, 0.224, 0.225])
7      ])
8
9      val_transform = transforms.Compose([
10         transforms.Resize((img_size, img_size)),
11         transforms.ToTensor(),
12         transforms.Normalize(mean=[0.485, 0.456, 0.406],
13                               std=[0.229, 0.224, 0.225])
14     ])
15
16     return train_transform, val_transform

```

Ta xây dựng từ điển câu trả lời (*answer vocabulary*) dựa trên thống kê các câu trả lời xuất hiện trong train/val, giới hạn tối đa số lớp để phục vụ bài toán phân loại:

```

1 def build_answer_vocab(train_data, val_data, max_answers=10000):
2     all_answers = []
3     for item in train_data:
4         all_answers.extend(item.get("answers", [item["answer"]]))
5     for item in val_data:
6         all_answers.extend(item.get("answers", [item["answer"]]))
7
8     answer_counts = Counter(all_answers)
9     unique_answers = list(answer_counts.keys())
10
11     if len(unique_answers) > max_answers:
12         unique_answers = [ans for ans, _ in answer_counts.most_common(max_answers)]
13
14     vocab = {ans: idx for idx, ans in enumerate(sorted(unique_answers))}
15
16     print(f"Built vocab: {len(vocab)} answers từ train+val (including all ground-truths
17           ")
18     return vocab

```

Tiếp theo, ta khai báo các siêu tham số chính dùng trong thực nghiệm (batch size, image size và số chiều TF-IDF):

```

1 data_dir = "data/vqav2"
2 batch_size = 32
3 img_size = 224
4 tfidf_dim = 1000

```

Đoạn code sau đọc dữ liệu train/val, xây dựng từ điển câu trả lời và thiết lập số lớp phân loại tương ứng:

```

1 train_transform, val_transform = get_transforms(img_size=img_size)
2
3 with open(os.path.join(data_dir, "train.json"), "r", encoding="utf-8") as f:
4     train_data = json.load(f)
5 with open(os.path.join(data_dir, "val.json"), "r", encoding="utf-8") as f:
6     val_data = json.load(f)
7
8 vocab = build_answer_vocab(train_data, val_data)
9 num_classes = len(vocab)
10 idx_to_ans = {idx: ans for ans, idx in vocab.items()}
11
12 print(f"Number of classes: {num_classes}")

```

Ta khởi tạo các tập train/val/test và đóng gói chúng thành DataLoader để phục vụ quá trình huấn luyện và đánh giá theo mini-batch:


```

1 train_dataset = VQADataset(
2     json_path=os.path.join(data_dir, "train.json"),
3     data_dir=data_dir,
4     transform=train_transform,
5     vocab=vocab
6 )
7
8 val_dataset = VQADataset(
9     json_path=os.path.join(data_dir, "val.json"),
10    data_dir=data_dir,
11    transform=val_transform,
12    vocab=vocab
13 )
14
15 test_dataset = VQADataset(
16     json_path=os.path.join(data_dir, "test.json"),
17     data_dir=data_dir,
18     transform=val_transform,
19     vocab=vocab
20 )
21
22 train_loader = DataLoader(train_dataset, batch_size=batch_size,
23                           shuffle=True, num_workers=2)
24 val_loader = DataLoader(val_dataset, batch_size=batch_size,
25                         shuffle=False, num_workers=2)
26 test_loader = DataLoader(test_dataset, batch_size=batch_size,
27                          shuffle=False, num_workers=2)

```

Ta fit bộ mã hoá TF-IDF trên toàn bộ câu hỏi trong tập train để trích xuất đặc trưng văn bản với số chiều cố định:

```

1 all_questions = []
2 for batch in train_loader:
3     all_questions.extend(batch["question"])
4 tfidf_vectorizer = TfidfVectorizer(max_features=tfidf_dim)
5 tfidf_vectorizer.fit(all_questions)
6
7 print(f"TFIDF vocabulary size: {len(tfidf_vectorizer.vocabulary_)}")

```

Đoạn code sau triển khai kiến trúc MLP baseline: ảnh được flatten, câu hỏi được mã hoá TF-IDF. Hai đặc trưng được nối lại và đưa qua bottleneck + classifier để dự đoán câu trả lời:

3. Xây dựng mô hình

```

1 class MLPBaseline(nn.Module):
2     def __init__(self, num_classes, img_size,
3                 tfidf_dim, tfidf_vectorizer, hidden_dim=512):
4         super().__init__()
5
6         self.img_size = img_size

```

```

7         self.tfidf_dim = tfidf_dim
8         self.img_flatten_dim = 3 * img_size * img_size
9         self.tfidf_vectorizer = tfidf_vectorizer
10
11         # Bottleneck: Fusion layer
12         fusion_dim = self.img_flatten_dim + tfidf_dim
13         self.bottleneck = nn.Sequential(
14             nn.Linear(fusion_dim, hidden_dim),
15             nn.ReLU(),
16             nn.Dropout(0.5)
17         )
18
19         # Fully Connected (FC): Classifier
20         self.classifier = nn.Sequential(
21             nn.Linear(hidden_dim, hidden_dim // 2),
22             nn.ReLU(),
23             nn.Dropout(0.5),
24             nn.Linear(hidden_dim // 2, num_classes)
25         )
26
27     def encode_questions(self, questions):
28         tfidf_features = self.tfidf_vectorizer.transform(questions).toarray()
29         return torch.FloatTensor(tfidf_features)
30
31     def forward(self, images, questions):
32         batch_size = images.size(0)
33
34         # Feature Extraction (FE)
35         img_features = images.view(batch_size, -1)
36         text_features = self.encode_questions(questions).to(images.device)
37
38         # Bottleneck: Fusion
39         combined = torch.cat([img_features, text_features], dim=1)
40         fused_features = self.bottleneck(combined)
41
42         # Fully Connected (FC): Classifier
43         output = self.classifier(fused_features)
44
45         return output

```

Ta xây dựng các hàm huấn luyện và đánh giá theo epoch. Trong đó, `train_epoch` tối ưu hoá Cross-Entropy trên các nhãn hợp lệ. Hàm `evaluate` ghi nhận các giá trị VQA top-1 accuracy, top-5 accuracy và average normalized Levenshtein similarity (ANLS) trên val/test. Đồng thời, ta triển khai khoảng cách Levenshtein và hàm ANLS để đánh giá độ tương đồng chuỗi dự đoán trong các trường hợp câu trả lời dạng văn bản:

4. Xây dựng hàm huấn luyện và đánh giá

```

1 def compute_vqa_accuracy(prediction, ground_truth_answers):
2     if not ground_truth_answers:
3         return 0.0

```

```

4
5     count = 0
6     pred = prediction.lower().strip()
7     for ans in ground_truth_answers:
8         if ans.lower().strip() == pred:
9             count += 1
10
11     return min(1.0, count / 3.0)
12
13 def levenshtein_distance(s1, s2):
14     if len(s1) > len(s2):
15         s1, s2 = s2, s1
16
17     distances = range(len(s1) + 1)
18     for i2, c2 in enumerate(s2):
19         distances_ = [i2+1]
20         for i1, c1 in enumerate(s1):
21             if c1 == c2:
22                 distances_.append(distances[i1])
23             else:
24                 distances_.append(1 + min((distances[i1], distances[i1 + 1], distances_
25                                         [-1])))
26
27         distances = distances_
28     return distances[-1]
29
30 def compute_anls(target, prediction):
31     target = target.lower().strip()
32     prediction = prediction.lower().strip()
33
34     if not target or not prediction:
35         return 0.0 if target != prediction else 1.0
36
37     dist = levenshtein_distance(target, prediction)
38     max_len = max(len(target), len(prediction))
39
40     score = 1.0 - float(dist) / float(max_len)
41     return score if score >= 0.5 else 0.0
42
43 def train_epoch(model, dataloader, criterion, optimizer, device, idx_to_ans):
44     model.train()
45     total_loss = 0
46     sum_vqa_acc = 0
47     total_samples = 0
48
49     for batch in dataloader:
50         images = batch["image"].to(device)
51         questions = batch["question"]
52         answers = batch["answer"].to(device)
53         raw_answers = batch["raw_answer"]
54         gt_answers_list = batch.get("answers", [[ans] for ans in raw_answers])
55
56         if isinstance(gt_answers_list, list) and len(gt_answers_list) > 0 and
57             isinstance(gt_answers_list[0], (list, tuple)
58             )):

```

```

55         gt_answers_list = list(zip(*gt_answers_list))
56
57         valid_mask = answers != -1
58         if not valid_mask.any():
59             continue
60
61         optimizer.zero_grad()
62         outputs = model(images, questions)
63
64         v_outputs = outputs[valid_mask]
65         v_answers = answers[valid_mask]
66         loss = criterion(v_outputs, v_answers)
67
68         loss.backward()
69         optimizer.step()
70
71         total_loss += loss.item() * v_answers.size(0)
72         batch_size = images.size(0)
73         total_samples += batch_size
74
75         _, predicted = outputs.max(1)
76         for i in range(batch_size):
77             pred_id = predicted[i].item()
78             pred_text = idx_to_ans.get(pred_id, "")
79             gt_list = gt_answers_list[i]
80             sum_vqa_acc += compute_vqa_accuracy(pred_text, gt_list)
81
82         avg_loss = total_loss / total_samples if total_samples > 0 else 0
83         vqa_acc = 100. * sum_vqa_acc / total_samples if total_samples > 0 else 0
84
85         return avg_loss, vqa_acc
86
87 def evaluate(model, dataloader, criterion, device, idx_to_ans):
88     model.eval()
89     total_loss = 0
90
91     correct_top1 = 0
92     correct_top5 = 0
93     total_valid_vocab = 0
94
95     sum_vqa_acc = 0
96     sum_vqa_top5 = 0
97
98     sum_anls = 0
99     total_samples = 0
100
101     with torch.no_grad():
102         for batch in dataloader:
103             images = batch["image"].to(device)
104             questions = batch["question"]
105             answers = batch["answer"].to(device)
106             raw_answers = batch["raw_answer"]
107             gt_answers_list = batch.get("answers", [[ans] for ans in raw_answers])
108

```

```

109         if isinstance(gt_answers_list, list) and len(gt_answers_list) > 0 and
110             isinstance(gt_answers_list[0], (list, tuple)):
111             gt_answers_list = list(zip(*gt_answers_list))
112
113         batch_size = images.size(0)
114         total_samples += batch_size
115
116         outputs = model(images, questions)
117
118         valid_mask = answers != -1
119         if valid_mask.any():
120             v_outputs = outputs[valid_mask]
121             v_answers = answers[valid_mask]
122             loss = criterion(v_outputs, v_answers)
123             total_loss += loss.item() * valid_mask.sum().item()
124
125             _, predicted = v_outputs.max(1)
126             correct_top1 += predicted.eq(v_answers).sum().item()
127
128             _, top5_preds = v_outputs.topk(min(5, v_outputs.size(1)), 1)
129             correct_top5 += (top5_preds == v_answers.view(-1, 1)).sum().item()
130
131             total_valid_vocab += valid_mask.sum().item()
132
133         # VQA Accuracy and ANLS
134         _, top5_all = outputs.topk(min(5, outputs.size(1)), 1)
135
136         for i in range(batch_size):
137             pred_id = top5_all[i, 0].item()
138             pred_text_top1 = idx_to_ans.get(pred_id, "")
139             gt_list = gt_answers_list[i]
140
141             # Top-1 VQA
142             sum_vqa_acc += compute_vqa_accuracy(pred_text_top1, gt_list)
143
144             # Top-5 VQA: max VQA score among top 5
145             vqa_scores_top5 = [compute_vqa_accuracy(idx_to_ans.get(top5_all[i, j].
146                 item(), ""), gt_list) for j in range(
147                 top5_all.size(1))]
148             sum_vqa_top5 += max(vqa_scores_top5)
149
150             # ANLS (Top-1)
151             sum_anls += compute_anls(raw_answers[i], pred_text_top1)
152
153         avg_loss = total_loss / total_valid_vocab if total_valid_vocab > 0 else 0
154         acc_top1 = 100. * correct_top1 / total_valid_vocab if total_valid_vocab > 0 else 0
155         acc_top5 = 100. * correct_top5 / total_valid_vocab if total_valid_vocab > 0 else 0
156
157         vqa_acc = 100. * sum_vqa_acc / total_samples if total_samples > 0 else 0
158         vqa_top5 = 100. * sum_vqa_top5 / total_samples if total_samples > 0 else 0
159         avg_anls = sum_anls / total_samples if total_samples > 0 else 0
160
161         return avg_loss, acc_top1, acc_top5, vqa_acc, vqa_top5, avg_anls

```

Từ các tham số đã thiết lập, ta khởi tạo mô hình MLP baseline với số lớp đầu ra tương ứng và chuyển mô hình sang thiết bị tính toán:

5. Cấu hình và khởi tạo mô hình

```
1 hidden_dim = 512
2
3 model = MLPBaseline(
4     num_classes=num_classes,
5     img_size=img_size,
6     tfidf_dim=tfidf_dim,
7     tfidf_vectorizer=tfidf_vectorizer,
8     hidden_dim=hidden_dim
9 ).to(device)
```

Ta cấu hình quá trình huấn luyện với Cross-Entropy loss và Adam optimizer, đồng thời chuẩn bị bộ nhớ để lưu lại lịch sử loss/accuracy theo từng epoch:

6. Huấn luyện mô hình

```
1 num_epochs = 10
2 lr = 0.001
3 criterion = nn.CrossEntropyLoss()
4 optimizer = torch.optim.Adam(model.parameters(), lr=lr)
5
6 history = {
7     "train_loss": [],
8     "train_acc": [],
9     "val_loss": [],
10    "val_acc": []
11 }
```

Đoạn code sau thực hiện vòng lặp huấn luyện theo epoch: huấn luyện trên train, đánh giá trên val, và lưu checkpoint có Acc(val) tốt nhất để phục vụ so sánh:

```
1 best_val_acc = 0
2 best_epoch = 0
3
4 for epoch in range(num_epochs):
5     train_loss, train_acc = train_epoch(model, train_loader,
6                                         criterion, optimizer, device)
7     val_loss, val_acc, val_top5, val_anls = evaluate(model, val_loader,
8                                                    criterion, device, idx_to_ans)
9
10    history["train_loss"].append(train_loss)
11    history["train_acc"].append(train_acc)
12    history["val_loss"].append(val_loss)
```

```

13     history["val_acc"].append(val_acc)
14
15     print(f"Epoch {epoch+1}/{num_epochs}")
16     print(f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%")
17     print(f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%, Val Top5: {val_top5:.2f}
18           %, Val ANLS: {val_anls:.4f}")
19
20     if val_acc > best_val_acc:
21         best_val_acc = val_acc
22         best_epoch = epoch + 1
23         torch.save({
24             "epoch": epoch,
25             "model_state_dict": model.state_dict(),
26             "optimizer_state_dict": optimizer.state_dict(),
27             "val_acc": val_acc,
28             "val_top5": val_top5,
29             "val_anls": val_anls,
30         }, "mlp_best.pth")
31     print(f"Saved best model with val acc: {val_acc:.2f}%")

```

Cuối cùng, ta đánh giá mô hình trên tập test và trực quan hoá đường học (training curves) cũng như kết quả tổng hợp của mô hình:

7. Đánh giá mô hình

```

1 test_loss, test_acc, test_top5, test_anls = evaluate(model, test_loader, criterion,
2                                                     device, idx_to_ans)
3 print(f"Test Loss: {test_loss:.4f}, Test Acc: {test_acc:.2f}%, Test Top5: {test_top5:.
4       2f}%, Test ANLS: {test_anls:.4f}")
5 plot_training_history(history, save_path="mlp_history.pdf", model_name="MLP")
6 plot_final_results(history["train_acc"][-1], best_val_acc, test_acc,
7                     save_path="mlp_results.pdf", model_name="MLP")

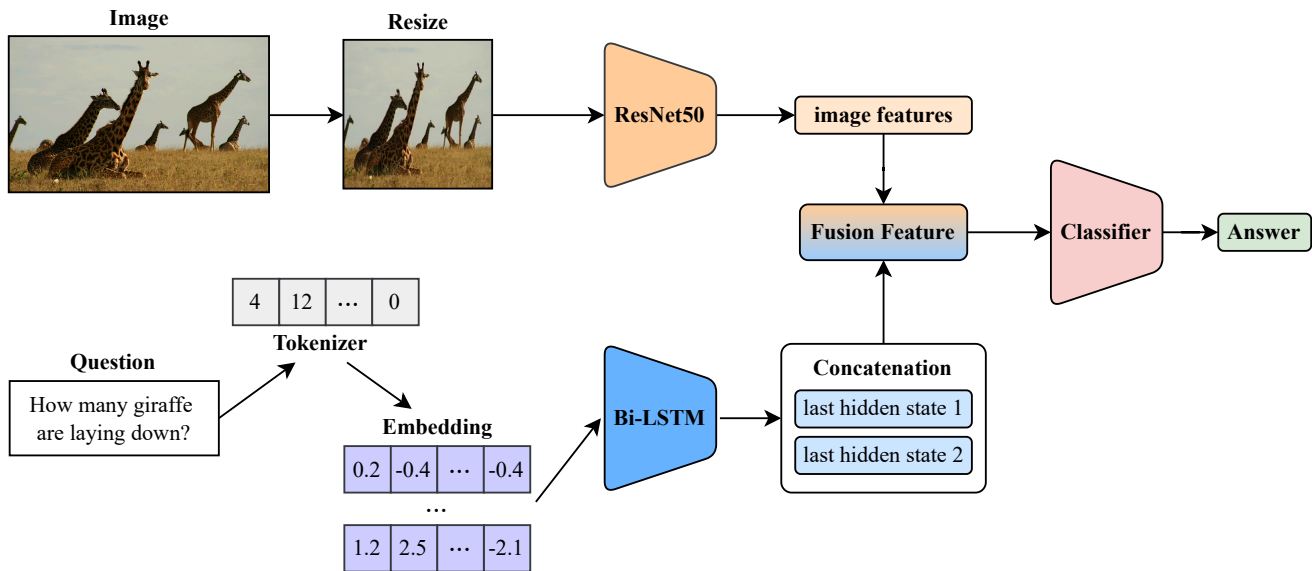
```

III.2. Kiến trúc 2: CNN+LSTM

Ở phần này, chúng ta triển khai hướng tiếp cận CNN+LSTM cho bài toán VQA. Khác với kiến trúc MLP ở phần trước (ảnh được flatten và câu hỏi dùng TF-IDF), mô hình này sử dụng hai nhánh encoder học sâu: trích xuất đặc trưng ảnh bằng CNN pretrained (ResNet50) và mã hoá câu hỏi bằng Embedding + BiLSTM. Hai đặc trưng sau đó được kết hợp bằng phép nối (concat) và đưa qua bottleneck cùng classifier để dự đoán câu trả lời.

Lưu ý: Những đoạn code được tái sử dụng ở III.1. sẽ không được nhắc lại, phần bên dưới chỉ đề cập các code cần điều chỉnh hoặc bổ sung.

Đoạn code sau bổ sung các thư viện cần thiết cho kiến trúc CNN+LSTM, trong đó `timm` được sử dụng để tải CNN pretrained (ResNet50):



Hình 16: Hình minh hoạ chi tiết mô hình VQA với CNN và LSTM.

1. Thiết lập môi trường

```

1 import json
2 import torch
3 import torch.nn as nn
4 from torch.utils.data import Dataset, DataLoader
5 from PIL import Image
6 import os
7 import timm
8 import torchvision.transforms as transforms
9 import random
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from collections import Counter

```

Ta giữ nguyên quy trình tải và dựng `DataLoader` như phần trước, đồng thời khai báo các tham số chính (batch size và image size) dùng cho thí nghiệm này:

2. Thiết lập bộ dữ liệu

```

1 data_dir = "data/vqav2"
2 batch_size = 32
3 img_size = 224

```

Để xây dựng từ vựng cho câu hỏi, ta gom toàn bộ câu hỏi từ tập train:

```

1 all_questions = []
2 for batch in train_loader:

```

```
3 all_questions.extend(batch["question"])
```

Tiếp theo, ta tạo từ điển từ (word-level) từ các câu hỏi huấn luyện, bao gồm hai token đặc biệt <PAD> và <UNK>, phục vụ cho bước tokenization và padding khi đưa vào LSTM:

```
1 word_set = set()
2 for q in all_questions:
3     words = q.lower().split()
4     word_set.update(words)
5
6 word_to_idx = {"<PAD>": 0, "<UNK>": 1}
7 for idx, word in enumerate(sorted(word_set), start=2):
8     word_to_idx[word] = idx
9
10 vocab_size = len(word_to_idx)
11 print(f"Vocabulary size: {vocab_size}")
```

Đoạn code sau triển khai mô hình CNN+LSTM: ảnh được trích xuất đặc trưng bằng ResNet50 pretrained (đóng băng tham số), câu hỏi được token hoá và mã hoá qua Embedding + BiLSTM; hai vector đặc trưng được concat, đi qua bottleneck và cuối cùng qua classifier để dự đoán nhãn câu trả lời:

3. Xây dựng mô hình

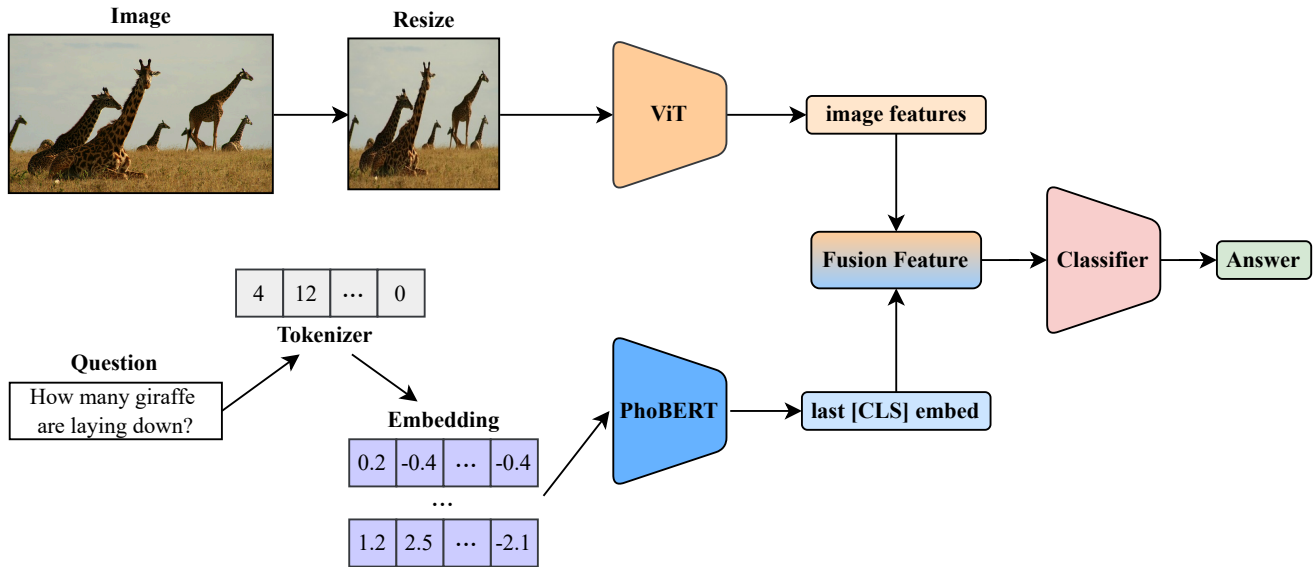
```
1 class CNNLSTMBaseline(nn.Module):
2     def __init__(self, num_classes, vocab_size, word_to_idx, hidden_dim=512,
3                   lstm_layers=2, embed_dim=300, max_len=64):
4
5         super().__init__()
6
7         # CNN encoder: ResNet pretrained
8         self.cnn = timm.create_model("resnet50", pretrained=True, num_classes=0)
9         # Freeze CNN backbone
10        for param in self.cnn.parameters():
11            param.requires_grad = False
12        cnn_output_dim = self.cnn.num_features
13
14        # Text encoder: Embedding + LSTM
15        self.embedding = nn.Embedding(vocab_size, embed_dim)
16        self.lstm = nn.LSTM(embed_dim, hidden_dim, lstm_layers,
17                            batch_first=True, bidirectional=True)
18
19        lstm_output_dim = hidden_dim * 2
20
21        # Bottleneck: Fusion layer
22        fusion_dim = cnn_output_dim + lstm_output_dim
23        self.bottleneck = nn.Sequential(
24            nn.Linear(fusion_dim, hidden_dim),
25            nn.ReLU(),
26            nn.Dropout(0.5))
```

```

25     )
26
27     # Fully Connected (FC): Classifier
28     self.classifier = nn.Linear(hidden_dim, num_classes)
29
30     # Store tokenizer
31     self.word_to_idx = word_to_idx
32     self.max_len = max_len
33
34     def tokenize(self, questions):
35         batch_tokens = []
36         for q in questions:
37             words = q.lower().split()
38             tokens = [self.word_to_idx.get(w, 1) for w in words]
39
40             # Padding
41             if len(tokens) < self.max_len:
42                 tokens = tokens + [0] * (self.max_len - len(tokens))
43             else:
44                 tokens = tokens[:self.max_len]
45
46             batch_tokens.append(tokens)
47
48         return torch.LongTensor(batch_tokens)
49
50     def forward(self, images, questions):
51         # Feature Extraction (FE)
52         # Image features: CNN encoding
53         img_features = self.cnn(images)
54
55         # Text features: LSTM encoding
56         question_tokens = self.tokenize(questions).to(images.device)
57         question_embeds = self.embedding(question_tokens)
58         lstm_out, (hidden, cell) = self.lstm(question_embeds)
59         # Last hidden state
60         text_features = torch.cat([hidden[-2], hidden[-1]], dim=1)
61
62         # Bottleneck: Fusion
63         combined = torch.cat([img_features, text_features], dim=1)
64         fused_features = self.bottleneck(combined)
65
66         # Fully Connected (FC): Classifier
67         output = self.classifier(fused_features)
68
69         return output

```

Cuối cùng, ta thiết lập các siêu tham số riêng cho nhánh ngôn ngữ (kích thước embedding, số tầng LSTM, độ dài chuỗi tối đa) và khởi tạo mô hình CNN+LSTM tương ứng:



Hình 17: Hình minh hoạ chi tiết mô hình VQA với ViT và BERT.

5. Cấu hình và khởi tạo mô hình

```

1 hidden_dim = 512
2 lstm_layers = 2
3 embed_dim = 300
4 max_len = 64
5
6 model = CNLSTMBaseline(
7     num_classes=num_classes,
8     vocab_size=vocab_size,
9     word_to_idx=word_to_idx,
10    hidden_dim=hidden_dim,
11    lstm_layers=lstm_layers,
12    embed_dim=embed_dim,
13    max_len=max_len
14 ).to(device)

```

Các bước huấn luyện và đánh giá được giữ nguyên như ở III.1..

III.3. Kiến trúc 3: ViT+BERT

Chúng ta triển khai hướng tiếp cận ViT+BERT cho bài toán VQA, trong đó ảnh được mã hoá bằng ViT pretrained và câu hỏi được mã hoá bằng BERT (trong trường hợp này là PhoBERT để encode dữ liệu tiếng Việt). Hai đặc trưng sau đó được kết hợp bằng phép nối (concat) và đưa qua bottleneck cùng classifier để dự đoán câu trả lời.

Lưu ý: Những đoạn code được tái sử dụng ở III.1. sẽ không được nhắc lại, phần bên dưới chỉ đề cập các code cần điều chỉnh hoặc bổ sung.

Đoạn code sau bổ sung các thư viện cần thiết cho kiến trúc ViT+BERT, bao gồm `timm` để tải

ViT pretrained và transformers để khởi tạo tokenizer/model BERT:

1. Thiết lập môi trường

```
1 import json
2 import torch
3 import torch.nn as nn
4 from torch.utils.data import Dataset, DataLoader
5 from PIL import Image
6 import os
7 import timm
8 from transformers import AutoTokenizer, AutoModel
9 import torchvision.transforms as transforms
10 import random
11 import numpy as np
12 import matplotlib.pyplot as plt
13 from collections import Counter
```

Ta giữ nguyên quy trình dựng dữ liệu như các phần trước, đồng thời khai báo cấu hình riêng cho nhánh ngôn ngữ, bao gồm tên mô hình BERT và độ dài chuỗi tối đa:

2. Thiết lập bộ dữ liệu

```
1 data_dir = "data/vqav2"
2 batch_size = 32
3 img_size = 224
4 bert_model = "vinai/phobert-base"
5 max_len = 64
6
7 print(f"BERT model: {bert_model}")
8 print(f"Image size: {img_size}")
```

Tiếp theo, ta khởi tạo tokenizer tương ứng để mã hoá câu hỏi đầu vào theo chuẩn của mô hình BERT đã chọn:

```
1 bert_tokenizer = AutoTokenizer.from_pretrained(bert_model)
2 print("BERT tokenizer loaded successfully")
```

Đoạn code sau triển khai kiến trúc ViT+BERT: ViT và BERT pretrained được dùng làm hai encoder (đóng băng tham số trong thiết lập mặc định), đặc trưng văn bản được lấy từ embedding của token [CLS], sau đó concat với đặc trưng ảnh và đưa qua bottleneck + classifier để phân loại câu trả lời:

3. Xây dựng mô hình

```
1 class ViTBERTBaseline(nn.Module):
2     def __init__(self, num_classes, bert_model, bert_tokenizer, hidden_dim=512,
                    img_size=224, max_len=64):
```

```

3     super().__init__()
4
5     # ViT encoder
6     self.vit = timm.create_model("vit_base_patch16_224", pretrained=True,
7                                 num_classes=0, img_size=img_size)
8     vit_output_dim = self.vit.num_features
9
10    # BERT encoder (Vietnamese)
11    self.tokenizer = bert_tokenizer
12    self.bert = AutoModel.from_pretrained(bert_model)
13    bert_output_dim = self.bert.config.hidden_size
14
15    for param in self.vit.parameters():
16        param.requires_grad = False
17    for param in self.bert.parameters():
18        param.requires_grad = False
19
20    # Bottleneck: Fusion layer
21    fusion_dim = vit_output_dim + bert_output_dim
22    self.bottleneck = nn.Sequential(
23        nn.Linear(fusion_dim, hidden_dim),
24        nn.ReLU(),
25        nn.Dropout(0.5)
26    )
27
28    # Fully Connected (FC): Classifier
29    self.classifier = nn.Linear(hidden_dim, num_classes)
30
31    self.max_len = max_len
32
33    def encode_questions(self, questions, device):
34        encoding = self.tokenizer(
35            questions,
36            padding=True,
37            truncation=True,
38            max_length=self.max_len,
39            return_tensors="pt"
40        )
41
42        input_ids = encoding["input_ids"].to(device)
43        attention_mask = encoding["attention_mask"].to(device)
44
45        # BERT encoding
46        outputs = self.bert(input_ids=input_ids, attention_mask=attention_mask)
47
48        # Lấy [CLS] token embedding
49        cls_embedding = outputs.last_hidden_state[:, 0, :]
50
51        return cls_embedding
52
53    def forward(self, images, questions):
54        # Feature Extraction (FE)
55        # Image features: ViT encoding
56        img_features = self.vit(images)

```

```
57
58     # Text features: BERT encoding
59     text_features = self.encode_questions(questions, images.device)
60
61     # Bottleneck: Fusion
62     combined = torch.cat([img_features, text_features], dim=1)
63     fused_features = self.bottleneck(combined)
64
65     # Fully Connected (FC): Classifier
66     output = self.classifier(fused_features)
67
68     return output
```

Cuối cùng, ta khởi tạo mô hình ViT+BERT với các tham số đã cấu hình và chuyển mô hình sang thiết bị tính toán.

```
1 hidden_dim = 512
2
3 model = ViTBERTBaseline(
4     num_classes=num_classes,
5     bert_model=bert_model,
6     bert_tokenizer=bert_tokenizer,
7     hidden_dim=hidden_dim,
8     img_size=img_size,
9     max_len=max_len
10 ).to(device)
11
12 print("Model initialized successfully")
```

IV. Thực nghiệm

IV.1. Thiết lập thực nghiệm

Trong phần thực nghiệm, ta tiến hành huấn luyện và đánh giá ba biến thể mô hình VQA đã được mô tả ở phần trên, gồm (1) MLP, (2) CNN+LSTM và (3) ViT+BERT trên hai bộ dữ liệu VQAv2 và ViInfographicVQA. Mỗi mô hình được huấn luyện trên tập train tương ứng của từng bộ dữ liệu và được đánh giá trên tập val/test nhằm so sánh hiệu năng theo từng kiến trúc, cũng như khảo sát tác động của các thành phần trong pipeline.

- **Môi trường thực nghiệm:** tất cả thực nghiệm được thực hiện trên một GPU NVIDIA A5000 24GB và hệ điều hành Ubuntu.
- **Thiết lập huấn luyện:** trừ khi được nêu cụ thể trong từng thí nghiệm, ta đóng băng các pretrained encoders và chỉ cập nhật tham số cho phần *prediction head* trong quá trình huấn luyện.
- **Độ đo đánh giá:** ta ghi nhận kết quả trên tập val/test theo các độ đo sau:
 - **Với VQAv2:** sử dụng VQA accuracy (V-Acc) và top-5 accuracy (Top-5 V-Acc), được trình bày theo định dạng phần trăm (%), trong khoảng 1-100).
 - **Với ViInfographicVQA:** sử dụng accuracy (Acc) truyền thống (được trình bày theo %, trong khoảng 1-100) và average normalized Levenshtein similarity (ANLS) score (trong khoảng 0-1).

Các giá trị được trình bày dưới dạng *Val/Test* tương ứng với tập validation và tập test của mỗi bộ dữ liệu. **Số được tô đậm** thể hiện kết quả cao thứ nhất, số được gạch chân thể hiện kết quả cao thứ hai.

IV.2. Độ đo đánh giá

Việc đánh giá một hệ thống VQA đòi hỏi các tiêu chuẩn khắt khe hơn so với các bài toán phân loại thông thường, do sự đa dạng trong cách diễn đạt ngôn ngữ tự nhiên và tính chủ quan của câu trả lời. Dự án này sử dụng hai độ đo chính tương ứng với đặc thù của từng bộ dữ liệu.

IV.2.1. VQA accuracy

Đối với bộ dữ liệu VQAv2 [3], độ đo tiêu chuẩn được thiết kế để xử lý sự bất đồng thuận giữa các người gán nhãn. Trong bộ dữ liệu này, mỗi câu hỏi đi kèm với 10 câu trả lời từ 10 người khác nhau. Một câu trả lời dự đoán được xem là chính xác nếu nó trùng khớp với sự đồng thuận của ít nhất 3 người. Công thức tính độ chính xác cho một câu hỏi được định nghĩa như sau:

$$\text{Acc}(\hat{a}) = \min \left(\frac{\sum_{i=1}^{10} \mathbb{I}(\hat{a} = g_i)}{3}, 1 \right) \quad (2)$$

Trong đó:

- $\hat{a}$: là câu trả lời do mô hình dự đoán.
- $G = \{g_1, g_2, \dots, g_{10}\}$: là tập hợp 10 câu trả lời tham chiếu, còn gọi là ground truth, từ con người.
- $\mathbb{I}(\cdot)$: là hàm chỉ thị, trả về 1 nếu điều kiện đúng và 0 nếu sai.

Độ đo này cho phép mô hình đạt điểm tối đa ngay cả khi không khớp với toàn bộ 10 người, miễn là đáp án đó được ít nhất 3 người đồng ý.

💡 Ví dụ tính toán VQA accuracy

Giả sử với hình ảnh một bầu trời và câu hỏi: “*Trời màu gì?*”.

Tập nhãn G gồm 10 câu trả lời: {“xanh”, “xanh”, “xanh”, “xanh dương”, “xanh dương”, “âm u”, ...}

- **Trường hợp 1:** Mô hình dự đoán $\hat{a} = \text{“xanh”}$.
Có 3 người trả lời “xanh”. Điểm số: $\min(\frac{3}{10}, 1) = 1.0$.
- **Trường hợp 2:** Mô hình dự đoán $\hat{a} = \text{“xanh dương”}$.
Có 2 người trả lời “xanh dương”. Điểm số: $\min(\frac{2}{10}, 1) \approx 0.67$.

IV.2.2. VQA accuracy top-5

Bên cạnh độ chính xác tiêu chuẩn, dự án còn sử dụng top-5 accuracy để đánh giá khả năng bao quát của mô hình trong không gian câu trả lời rộng lớn. Trong bài toán phân loại với hàng nghìn nhãn khác nhau thì đáp án chính xác có thể không phải lúc nào cũng đạt điểm số cao nhất nhưng thường xuất hiện trong nhóm dẫn đầu.

Với độ đo này, một mẫu dữ liệu được tính là dự đoán đúng nếu nhãn mục tiêu g nằm trong tập hợp 5 câu trả lời có xác suất dự đoán cao nhất $\mathcal{K}_5$. Công thức tính cho một mẫu dữ liệu là:

$$\text{Acc@5} = \mathbb{I}(g \in \mathcal{K}_5) \quad (3)$$

Trong đó $\mathcal{K}_5$ là tập hợp 5 nhãn lớp có giá trị logit lớn nhất ở đầu ra của mô hình. Độ đo này giúp nhận diện các trường hợp mô hình hiểu đúng ngữ cảnh nhưng bị nhầm lẫn nhỏ giữa các từ đồng nghĩa hoặc các khái niệm gần gũi.

💡 Ví dụ tính toán Top-5 accuracy

Câu hỏi: “*Con vật trong hình là gì?*”. Nhãn đúng: $g = \text{“chó”}$.

Phân bố dự đoán của mô hình xếp theo thứ tự giảm dần:

1. “mèo” (0.45)
2. “chó” (0.30)
3. “hổ” (0.10)

4. “sư tử” (0.05)

5. “gấu” (0.05)

Kết quả:

- **VQA accuracy (top-1):** Dự đoán cao nhất là “mèo” khác với nhãn “chó”. Điểm số: 0.
- **Top-5 accuracy:** Tập 5 dự đoán đầu bao gồm “chó” trùng khớp với nhãn. Điểm số: 1.

IV.2.3. Average normalized Levenshtein similarity (ANLS)

Đối với bộ dữ liệu ViInfographicVQA, câu trả lời thường là các chuỗi ký tự được trích xuất từ ảnh. Do các vấn đề về lỗi nhận dạng ký tự hoặc sự khác biệt nhỏ về định dạng, các độ đo có cơ chế exact match thường quá khắt khe. Do đó, dự án sử dụng độ đo ANLS [11] để đánh giá sự tương đồng, đây là độ đo thường dùng đánh giá trong các bài toán dạng generative.

ANLS dựa trên khoảng cách chỉnh sửa Levenshtein. Đầu tiên, điểm số normalized Levenshtein similarity (NL) giữa dự đoán $\hat{a}$ và một nhãn g được tính như sau:

$$NL(\hat{a}, g) = 1 - \frac{d_L(\hat{a}, g)}{\max(|\hat{a}|, |g|)} \quad (4)$$

Trong đó $d_L(\hat{a}, g)$ là số thao tác thay đổi tối thiểu (thêm, xóa, sửa) để biến chuỗi này thành chuỗi kia, và $|\cdot|$ là độ dài của chuỗi.

Điểm số ANLS cuối cùng cho tập dữ liệu N mẫu là trung bình cộng của các điểm NL tốt nhất, có áp dụng ngưỡng lọc $\tau = 0.5$:

$$ANLS = \frac{1}{N} \sum_{i=1}^N \max_{g \in G_i} (s(NL(\hat{a}_i, g))) \quad (5)$$

Hàm ngưỡng $s(x)$ trả về giá trị x nếu $x \geq \tau$, ngược lại trả về 0. Điều này có nghĩa là nếu dự đoán sai quá 50% độ dài chuỗi, điểm số sẽ bị phạt về 0.



Ví dụ tính toán ANLS

Câu trả lời đúng: $g = \text{“Vietnam”}$ (độ dài 7).

- **Trường hợp 1:** Dự đoán $\hat{a} = \text{“Vietna”}$ (thiếu chữ ‘m’).

Khoảng cách $d_L = 1$. Độ dài lớn nhất = 7.

$NL = 1 - 1/7 \approx 0.86$. Vì $0.86 \geq 0.5$, điểm số là **0.86**.

- **Trường hợp 2:** Dự đoán $\hat{a} = \text{“V1et12n”}$.

Khoảng cách $d_L = 4$ (4 phép thay thế: ‘i’→‘1’, ‘n’→‘1’, ‘a’→‘2’, ‘m’→‘n’). Độ dài

lớn nhất = 7.

NL = $1 - 4/7 \approx 0.43$. Vì $0.43 < 0.5$, điểm số bị phạt về **0.0**.

IV.3. Kết quả chính

Trong thực nghiệm chính, ta chọn batch size $\in \{16, 32\}$, image size $\in \{224, 448\}$ và learning rate = $1e-3$. Các pretrained model được sử dụng đều được đồng bộ tham số, nghĩa là không được cập nhật gradient trong quá trình huấn luyện. Toàn bộ mô hình được huấn luyện với 10 epoch. Kết quả thu được như sau:

Bảng 2: Kết quả đánh giá của ba biến thể mô hình trên tập val/test của VQAv2 với các cấu hình batch size và image size khác nhau.

Architecture	Batch	Image	Epoch	LR	V-Acc		Top-5 V-Acc	
					Val	Test	Val	Test
MLP	16	224	10	$1e-3$	3.10	2.20	10.33	8.33
CNN+LSTM	16	224	10	$1e-3$	52.77	54.70	83.57	<u>82.80</u>
ViT+BERT	16	224	10	$1e-3$	44.50	47.87	73.33	76.23
MLP	16	448	10	$1e-3$	3.10	1.53	11.67	6.63
CNN+LSTM	16	448	10	$1e-3$	<u>53.13</u>	54.07	82.23	80.53
ViT+BERT	16	448	10	$1e-3$	45.30	48.63	74.80	76.17
MLP	32	224	10	$1e-3$	2.97	1.30	9.60	6.70
CNN+LSTM	32	224	10	$1e-3$	53.43	56.23	<u>82.43</u>	84.93
ViT+BERT	32	224	10	$1e-3$	45.57	46.80	74.60	77.33
MLP	32	448	10	$1e-3$	2.73	1.30	9.87	7.03
CNN+LSTM	32	448	10	$1e-3$	52.27	<u>56.07</u>	<u>82.43</u>	81.37
ViT+BERT	32	448	10	$1e-3$	46.13	47.93	74.47	76.37

Bảng 3: Kết quả đánh giá của ba biến thể mô hình trên tập val/test của ViInfographicVQA với các cấu hình batch size và image size khác nhau.

Architecture	Batch	Image	Epoch	LR	Acc		ANLS	
					Val	Test	Val	Test
MLP	16	224	10	$1e-3$	1.90	0.63	0.0400	0.0140
CNN+LSTM	16	224	10	$1e-3$	<u>6.40</u>	3.17	0.0849	0.0893
ViT+BERT	16	224	10	$1e-3$	4.10	1.69	0.0549	0.0304
MLP	16	448	10	$1e-3$	2.00	0.32	0.0270	0.0175

Continued on next page

Architecture	Batch	Image	Epoch	LR	Acc		ANLS	
					Val	Test	Val	Test
CNN+LSTM	16	448	10	$1e-3$	5.40	3.91	0.0714	0.0615
ViT+BERT	16	448	10	$1e-3$	3.80	1.90	0.0513	0.0422
MLP	32	224	10	$1e-3$	1.80	0.63	0.0420	0.0140
CNN+LSTM	32	224	10	$1e-3$	6.80	4.86	0.0876	0.0893
ViT+BERT	32	224	10	$1e-3$	4.20	3.48	0.0561	0.0461
MLP	32	448	10	$1e-3$	1.90	0.63	0.0255	0.0145
CNN+LSTM	32	448	10	$1e-3$	5.90	4.86	0.0757	0.0981
ViT+BERT	32	448	10	$1e-3$	4.40	2.85	0.0513	0.0429

Theo kết quả ở Bảng 2 và Bảng 3, ta có thể rút ra một số quan sát sau:

- **Về mặt kiến trúc:** CNN+LSTM cho hiệu năng tốt và ổn định nhất trên cả hai bộ dữ liệu. ViT+BERT đạt mức trung bình nhưng không vượt CNN+LSTM, đây là một dữ kiện bất ngờ khi thiết kế dựa trên các mô hình họ Transformers lại không đạt kết quả cao nhất. MLP thấp nhất, cho thấy biểu diễn đặc trưng đơn giản không phù hợp cho bài toán VQA ở cả ảnh tự nhiên lẫn infographic.
- **Trên VQAv2:** CNN+LSTM vượt trội so với các baseline còn lại. Cấu hình tốt nhất đạt Accuracy(Test)=**56.23%** và Top-5(Test)=**84.93%** (Batch=32, Image=224). ViT+BERT đạt Accuracy(Test) khoảng **46.80-48.63%** và Top-5(Test) khoảng **76.17-77.33%**, cao hơn MLP nhưng vẫn kém CNN+LSTM. MLP chỉ đạt Accuracy(Test) khoảng **1.30-2.20%**.
- **Trên ViInfographicVQA:** Nhìn chung, accuracy của tất cả mô hình giảm mạnh so với VQAv2. Kết quả tốt nhất thuộc về CNN+LSTM với Accuracy(Test)=**4.86%** (Batch=32, Image=448). Xét theo ANLS, CNN+LSTM tiếp tục dẫn đầu với ANLS(Test)=**0.0981**, trong khi ViT+BERT đạt **0.0304-0.0461** và MLP đạt **0.0140-0.0175**, cho thấy bộ dữ liệu ViInfographicVQA là bài toán khó hơn rõ rệt.

Bảng 4: Kết quả đánh giá của ba biến thể mô hình trên tập val/test sau khi được huấn luyện trên tập train của hai bộ dữ liệu VQAv2 và ViInfographic. Kiến trúc ViT+BERT được huấn luyện trên các giá trị siêu tham số mới cho ra kết quả tốt hơn.

Architecture	VQAv2				ViInfographicVQA			
	V-Acc		Top-5 V-Acc		Acc		ANLS	
	Val	Test	Val	Test	Val	Test	Val	Test
MLP (best)	1.70	0.70	8.10	3.20	2.00	0.32	0.0270	0.0175
CNN+LSTM (best)	<u>50.40</u>	<u>54.10</u>	<u>79.50</u>	<u>80.80</u>	<u>5.90</u>	<u>4.86</u>	<u>0.0757</u>	<u>0.0981</u>
ViT+BERT (new best)	58.40	60.73	85.00	84.63	9.90	18.16	0.1579	0.2601

Mở rộng nhỏ: Với kết quả trên, rõ ràng kiến trúc CNN+LSTM có kết quả tốt hơn ViT+BERT trên bài toán này. Tuy nhiên, liệu rằng điều có thật sự luôn đúng hay không? Câu hỏi này dẫn

nhóm đến việc thử huấn luyện lại mô hình này với cài đặt siêu tham số khác nhau. Mô hình được lần lượt điều chỉnh lại, bao gồm mở bằng toàn bộ tham số mô hình để huấn luyện; cài đặt giá trị learning rate riêng cho phần ViT, BERT là $1e - 5$ và MLP trong classifier là $1e - 3$. Các tham số khác như batch size = 32, epoch = 10, image size = 224×224 vẫn được giữ nguyên cho cả ba mô hình.

Theo ghi nhận ở Bảng 4, ta có thể quan sát thấy rằng với sự điều chỉnh siêu tham số huấn luyện phù hợp, kiến trúc kết hợp giữa ViT+BERT sẽ mang lại hiệu năng tốt hơn, đúng với kỳ vọng về mô hình thuộc họ Transformers. Điều này cho thấy, huấn luyện mô hình dữ liệu đa phương thức như VQA, đòi hỏi chúng ta phải thực hiện điều chỉnh siêu tham số và setup mô hình một cách phù hợp. Đây cũng là công việc thường thấy trong hầu hết các bài toán học máy khác.

IV.4. Thực nghiệm về các cách kết hợp đặc trưng

Trong các kiến trúc VQA theo hướng tiếp cận *fusion*, một công đoạn quan trọng là *kết hợp đặc trưng* giữa ảnh và văn bản để mô hình có thể học được tương tác đa phương thức - mối liên hệ giữa hai nguồn thông tin ảnh và văn bản. Vì vậy, trong phần này, ta tiến hành khảo sát bốn cách thiết kế fusion giữa hai vector đặc trưng thu được từ image encoder ($\mathbf{v}_{img}$) và text encoder ($\mathbf{v}_{txt}$). Cụ thể như sau:

- **Concat:** Nối trực tiếp hai vector đặc trưng $\mathbf{v}_{img}$ và $\mathbf{v}_{txt}$ theo chiều kênh để tạo vector hợp nhất:

$$\mathbf{z} = [\mathbf{v}_{img}; \mathbf{v}_{txt}].$$

- **Linear→Concat:** Thực hiện linear projection mỗi vector về cùng số chiều, sau đó nối:

$$\tilde{\mathbf{v}}_{img} = W_{img}\mathbf{v}_{img}, \quad \tilde{\mathbf{v}}_{txt} = W_{txt}\mathbf{v}_{txt}, \quad \mathbf{z} = [\tilde{\mathbf{v}}_{img}; \tilde{\mathbf{v}}_{txt}].$$

- **Linear→Plus:** Chiếu tuyến tính về cùng số chiều, sau đó cộng theo phần tử để trộn thông tin hai modality:

$$\tilde{\mathbf{v}}_{img} = W_{img}\mathbf{v}_{img}, \quad \tilde{\mathbf{v}}_{txt} = W_{txt}\mathbf{v}_{txt}, \quad \mathbf{z} = \tilde{\mathbf{v}}_{img} + \tilde{\mathbf{v}}_{txt}.$$

- **Linear→Multiply:** Chiếu tuyến tính về cùng số chiều, sau đó nhân theo phần tử (có thể hiểu như một dạng *gating* giữa hai modality):

$$\tilde{\mathbf{v}}_{img} = W_{img}\mathbf{v}_{img}, \quad \tilde{\mathbf{v}}_{txt} = W_{txt}\mathbf{v}_{txt}, \quad \mathbf{z} = \tilde{\mathbf{v}}_{img} \odot \tilde{\mathbf{v}}_{txt}.$$

Bảng 5: Kết quả đánh giá các chiến lược kết hợp đặc trưng (fusion) trên tập val/test của VQAv2 và ViInfographicVQA với batch size = 32, image size = 224, epoch = 10, lr = $1e-3$. Với phép cộng/nhân, đặc trưng được *linear projection* về cùng chiều trước khi thực hiện phép toán.

Architecture	Fusion	VQAv2				ViInfographicVQA			
		V-Acc		Top-5 V-Acc		Acc		ANLS	
		Val	Test	Val	Test	Val	Test	Val	Test
MLP	Concat	2.97	1.30	9.60	6.70	1.80	0.63	0.0420	0.0140
CNN+LSTM	Concat	<u>53.43</u>	56.23	<u>82.43</u>	84.93	<u>6.80</u>	4.86	0.0876	<u>0.0893</u>
ViT+BERT	Concat	45.57	46.80	74.60	77.33	4.20	3.48	0.0561	0.0461
MLP	Linear→Concat	2.60	1.63	8.50	7.60	2.00	0.32	0.0492	0.0175
CNN+LSTM	Linear→Concat	53.63	<u>54.73</u>	81.33	<u>81.17</u>	6.30	5.70	<u>0.0971</u>	0.0877
ViT+BERT	Linear→Concat	46.90	49.93	75.93	76.53	4.70	2.64	0.0614	0.0491
MLP	Linear→Plus	2.90	1.93	9.20	9.53	1.40	0.32	0.0195	0.0040
CNN+LSTM	Linear→Plus	52.47	53.43	83.00	80.97	6.00	4.75	0.0780	0.0850
ViT+BERT	Linear→Plus	44.43	47.77	71.77	76.00	4.00	2.01	0.0544	0.0372
MLP	Linear→Multiply	20.83	17.27	48.47	45.17	2.70	0.74	0.0412	0.0165
CNN+LSTM	Linear→Multiply	49.73	50.10	78.47	77.13	6.90	<u>5.39</u>	0.1045	0.1108
ViT+BERT	Linear→Multiply	47.53	48.63	74.30	76.17	4.80	1.90	0.0651	0.0392

Theo kết quả ở Bảng 5, ta có thể rút ra một số quan sát sau:

- **Xu hướng tổng quát:** fusion ảnh hưởng mạnh đến hiệu năng, tuy nhiên tác động *không đồng nhất* theo kiến trúc. Các phép *element-wise* (đặc biệt là *Multiply*) đem lại lợi ích rất rõ cho mô hình đơn giản (MLP), trong khi với các mô hình mạnh hơn (CNN+LSTM, ViT+BERT) thì *Concat/Linear→Concat* thường ổn định và cạnh tranh hơn.
- **Trên VQAv2:** (i) Với *CNN+LSTM*, *Concat* đạt tốt nhất trên tập test với V-Acc=**56.23%** và Top-5=**84.93%**. Các phép *element-wise* (Plus/Multiply) không vượt được *Concat*. (ii) Với **ViT+BERT**, *Linear→Concat* cho V-Acc(Test) cao nhất **49.93%**, trong khi Top-5(Test) tốt nhất vẫn là *Concat* với **77.33%**. Điều này gợi ý rằng việc thêm lớp tuyến tính trước khi nối đặc trưng giúp mô hình tận dụng tốt hơn không gian biểu diễn, nhưng không hoàn toàn cải thiện top-5 accuracy.
- **Trên ViInfographicVQA:** với *CNN+LSTM*, *Multiply* cho ANLS(Test) cao nhất đạt **0.1108**, trong khi *Linear→Concat* đạt Acc(Test) tốt nhất **5.70%**. Kết quả này cho thấy *Multiply* có thể đóng vai trò như một cơ chế tương tác/gating theo từng chiều đặc trưng, hữu ích cho thước đo dựa trên độ tương đồng chuỗi (ANLS) trong bối cảnh infographic.
- **Nhận xét thêm (MLP hưởng lợi rất mạnh từ Multiply):** trên *VQAv2*, *Linear→Multiply* giúp V-Acc(Test) tăng từ **1.30%** (*Concat*) lên **17.27%**, đồng thời Top-5(Test) tăng từ **6.70%** lên **45.17%**. Đây là mức cải thiện vượt trội so với các lựa chọn fusion khác, cho thấy khi năng lực mô hình hạn chế, fusion dạng nhân có thể hoạt động như một cơ chế chọn lọc đặc trưng hiệu quả. Trên **ViInfographicVQA**, *Multiply* cũng cho Acc(Test) tốt nhất

của MLP (**0.74%**), nhưng ANLS(Test) cao nhất lại thuộc về Linear→Concat (**0.0175**), cho thấy tối ưu Acc không nhất thiết đồng pha với tối ưu ANLS.

IV.5. Thực nghiệm về thiết kế classifier

Trong các kiến trúc VQA, sau khi thực hiện fusion giữa đặc trưng ảnh và văn bản, một bước quan trọng tiếp theo là *classifier* phải đủ năng lực để hiểu được biểu diễn hợp nhất này và ánh xạ về không gian câu trả lời (answer space). Từ thực nghiệm fusion ở phần trước, phép *Concat* được ghi nhận là một trong các chiến lược hiệu quả nhất. Tuy nhiên, khi hai đặc trưng được nối lại tạo thành một vector mới có chiều lớn hơn và giàu thông tin hơn, ta kỳ vọng kiến trúc *post-fusion classifier* sẽ ảnh hưởng đáng kể đến khả năng khai thác biểu diễn này. Vì vậy, trong thực nghiệm này, ta khảo sát ba thiết kế classifier sau fusion:

- **Post-fusion: None (Baseline).** Vector fusion $\mathbf{z} = [\mathbf{v}_{img}; \mathbf{v}_{txt}]$ được đưa trực tiếp qua tầng phân loại (ví dụ MLP/linear head) mà không thêm khối xử lý tuần tự hay cơ chế chú ý sau fusion. Đây là thiết lập chuẩn để so sánh.
- **Post-fusion: LSTM.** Xem vector fusion như một chuỗi (hoặc chia thành các “token” theo chiều đặc trưng) và đưa qua một tầng LSTM nhằm mô hình hoá tương tác/quan hệ phụ thuộc trong biểu diễn hợp nhất trước khi phân loại. Thiết kế này kỳ vọng tăng khả năng khai thác cấu trúc ẩn trong $\mathbf{z}$.
- **Post-fusion: Attention.** Áp dụng một khối attention sau fusion để học cơ chế gán trọng số lên các thành phần trong vector hợp nhất, từ đó nhấn mạnh thông tin quan trọng và giảm nhiễu trước khi đưa vào classifier.

Bảng 6: Kết quả đánh giá các thiết kế *post-fusion classifier* với fusion cố định là *Concat* và cùng thiết lập huấn luyện (batch size = 32, image size = 224, epoch = 10, lr = $1e-3$).

Architecture	Post-Fusion	VQAv2				ViInfographicVQA			
		V-Acc		Top-5 V-Acc		Acc		ANLS	
		Val	Test	Val	Test	Val	Test	Val	Test
MLP	None	2.97	1.30	9.60	6.70	1.80	0.63	0.0420	0.0140
CNN+LSTM	None	<u>53.43</u>	56.23	<u>82.43</u>	84.93	6.80	<u>4.86</u>	0.0876	0.0893
ViT+BERT	None	45.57	46.80	74.60	77.33	4.20	3.48	0.0561	0.0461
MLP	LSTM	5.90	6.40	17.90	20.17	1.50	0.63	0.0235	0.0140
CNN+LSTM	LSTM	50.43	51.67	79.83	79.30	6.00	4.33	0.0985	<u>0.0953</u>
ViT+BERT	LSTM	43.87	47.10	73.33	74.57	3.50	1.80	0.0443	0.0378
MLP	Attention	3.33	2.27	8.90	7.90	1.80	0.32	0.0185	0.0165
CNN+LSTM	Attention	53.60	<u>54.27</u>	82.90	<u>83.03</u>	<u>6.20</u>	6.02	<u>0.0961</u>	0.1101
ViT+BERT	Attention	45.80	49.37	76.03	76.87	5.10	3.59	0.0632	0.0508

Theo kết quả ở Bảng 6, ta có thể rút ra một số quan sát sau:

- **Tổng quát:** thiết kế *post-fusion* ảnh hưởng đáng kể tới chất lượng mô hình, đặc biệt trên ViInfographicVQA (nơi cần cơ chế chọn lọc/nhấn mạnh thông tin sau khi concat).
- **Trên VQAv2:** với CNN+LSTM, thêm *attention* sau fusion cho kết quả tốt nhất trên tập Val với $\text{Acc(Val)}=53.60\%$ và $\text{Top-5(Val)}=82.90\%$, tăng so với baseline None lần lượt là 53.43% và 82.43%. Tuy nhiên, kết quả trên tập Test tốt nhất vẫn thuộc về baseline None với $\text{Acc(Test)}=56.23\%$ và $\text{Top-5(Test)}=84.93\%$, cho thấy cải thiện trên tập Val không đồng nghĩa kết quả tăng trên tập Test.
- **Trên ViInfographicVQA:** *attention* sau fusion mang lại hiệu quả rõ nhất với CNN+LSTM, đạt $\text{Acc(Test)}=6.02\%$ và $\text{ANLS(Test)}=0.1101$ (cao hơn đáng kể so với baseline None: 4.86% và 0.0893). Điều này phù hợp với trực giác rằng infographic đòi hỏi cơ chế tập trung vào thành phần quan trọng trong vector fusion.
- **Theo kiến trúc:** ViT+BERT chỉ cải thiện nhẹ khi thêm *attention* (VQAv2 Acc(Test) 49.37% so với 46.80%), và nhìn chung vẫn chưa vượt CNN+LSTM trong thiết lập hiện tại. Với MLP, thêm LSTM giúp tăng mạnh trên VQAv2 (Acc(Test) 6.40% so với 1.30% và Top-5(Test) 20.17% so với 6.70%) nhưng vẫn kém xa hai kiến trúc còn lại.

Thông qua các kết quả thu được từ phần thực nghiệm, có thể thấy bài toán VQA là một trong những bài toán có thách thức lớn với các mô hình deep learning truyền thống. Kết quả cho thấy hiệu năng không chỉ phụ thuộc vào backbone, mà còn chịu ảnh hưởng mạnh bởi thiết lập huấn luyện, chiến lược fusion và thiết kế classifier sau fusion. Ở thiết lập mặc định, CNN+LSTM cho kết quả ổn định nhất. Tuy nhiên, khi điều chỉnh các thiết lập trong huấn luyện mô hình một cách phù hợp (mở băng tham số và đặt learning rate riêng), ViT+BERT cải thiện rõ rệt và đạt kết quả tốt hơn, đúng với kỳ vọng về các mô hình thuộc họ Transformers.

Bên cạnh đó, Concat (và biến thể Linear→Concat) là chiến lược fusion hiệu quả và ổn định trên nhiều thiết lập, đặc biệt khi đánh giá theo accuracy. Trong khi đó, các phép toán theo phần tử như Multiply cho thấy lợi ích nổi bật với mô hình đơn giản như MLP trên VQAv2, và có thể cải thiện đáng kể theo thước đo ANLS trên ViInfographicVQA. Ngoài ra, các khối post-fusion như LSTM/attention có thể mang lại cải thiện bổ sung (đặc biệt là phép attention trên ViInfographicVQA), nhưng hiệu quả phụ thuộc vào kiến trúc và thước đo đánh giá. Tổng quan, các quan sát này nhấn mạnh rằng để đạt hiệu năng tốt trong VQA, cần tối ưu đồng thời nhiều thành phần trong pipeline thay vì chỉ thay đổi kiến trúc đơn lẻ.

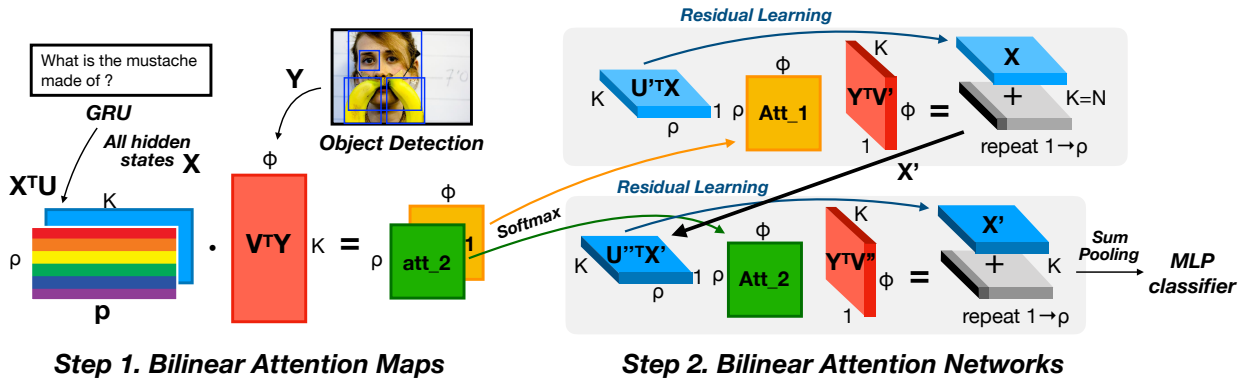
V. Các hướng mở rộng

Dựa trên những kết quả thực nghiệm và hạn chế của các phương pháp baselines hiện tại, một số hướng nghiên cứu tiềm năng dưới đây có thể được áp dụng để cải thiện hiệu suất và khả năng của hệ thống.

1. Cải thiện qua cơ chế attention

Tương tự như kiến trúc UpDn đã được nhắc đến ở trên, việc biểu diễn và kết hợp đặc trưng hình ảnh và câu hỏi đóng vai trò then chốt. Để khắc phục nhược điểm của phép nối vector đơn giản, các nghiên cứu dưới đây tập trung khai thác sự tương tác giữa hai luồng dữ liệu thông qua các cơ chế attention phức tạp hơn:

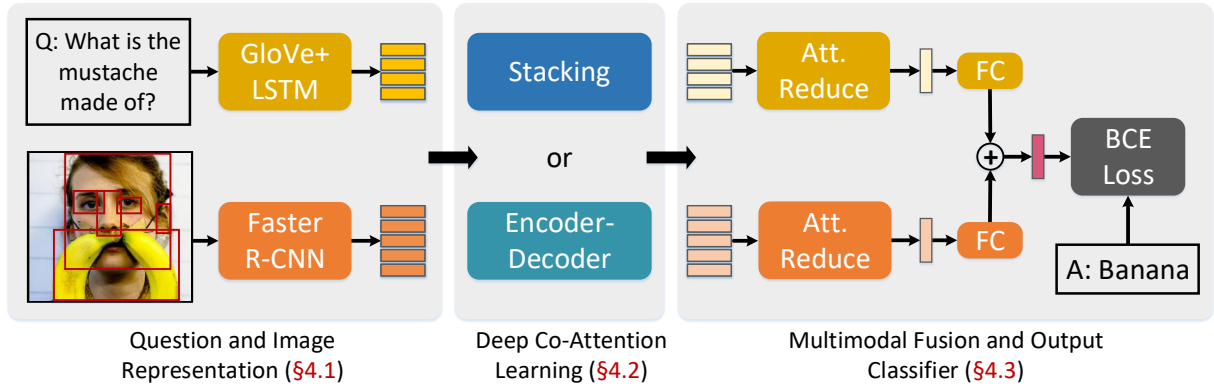
- **Bilinear attention networks (BAN)** [12]: thay vì chỉ tương tác một chiều hoặc sử dụng đặc trưng trung bình, BAN học các mối quan hệ phức tạp giữa mọi vùng ảnh và mọi từ trong câu hỏi, được minh họa trong Hình 18.



Hình 18: Kiến trúc BAN sử dụng bilinear attention maps kết hợp với residual learning. Ảnh trích từ [12].

Cụ thể, mô hình sử dụng phân phối bilinear kết hợp với kỹ thuật giảm chiều dữ liệu low-rank bilinear pooling. Điều này cho phép tận dụng thông tin từ cả hai luồng dữ liệu mà không làm tăng đáng kể số lượng tham số, đồng thời sử dụng cơ chế residual learning để tăng cường khả năng biểu diễn.

- **Deep modular co-attention networks (DCAN)** [13]: kiến trúc này mở rộng khả năng suy luận bằng cách stacking nhiều lớp attention theo chiều sâu, thay vì chỉ sử dụng một lớp nông như các mô hình truyền thống, được trực quan trong Hình 19.



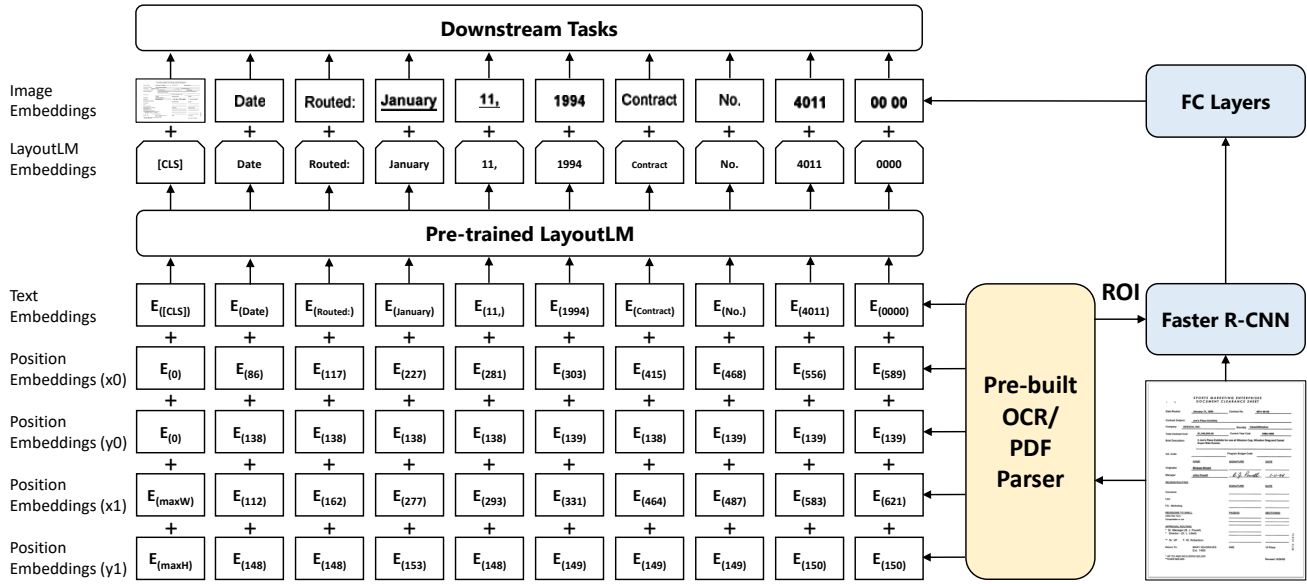
Hình 19: Quy trình deep co-attention learning trong DCAN với các khối xếp chồng để tinh chỉnh đặc trưng. Ảnh trích từ [13].

DCAN mô hình hóa đồng thời sự tương tác nội bộ và tương tác chéo giữa hình ảnh và văn bản. Thông qua kiến trúc module dạng encoder-decoder hoặc stacking, đặc trưng của cả hai luồng được tinh chỉnh lặp đi lặp lại qua nhiều lớp, giúp mô hình nắm bắt được ngữ cảnh phân cấp và mối quan hệ ngữ nghĩa sâu sắc hơn.

Việc tích hợp các cơ chế này hứa hẹn sẽ giúp mô hình không chỉ định vị chính xác các vùng hình ảnh liên quan mà còn hiểu rõ hơn cấu trúc câu hỏi phức tạp.

2. Tích hợp khả năng đọc hiểu văn bản trong ảnh

Đối với các bài toán như Infographic VQA hay TextVQA, nơi câu trả lời nằm trực tiếp trong nội dung văn bản của hình ảnh, các mô hình VQA truyền thống thường gặp khó khăn do thiếu khả năng hiểu cấu trúc tài liệu. Các kỹ thuật chuyên biệt liên quan đến document cần được tích hợp để giải quyết vấn đề này:



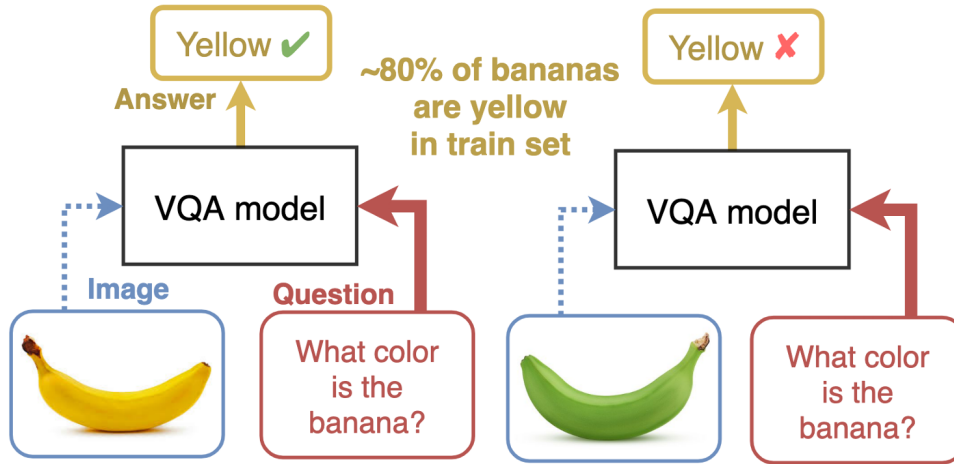
Hình 20: Kiến trúc LayoutLM kết hợp đặc trưng văn bản, tọa độ không gian 2D và đặc trưng thị giác. Ảnh trích từ [14].

Khác với các mô hình ngôn ngữ truyền thống chỉ xử lý văn bản theo chuỗi tuyến tính, LayoutLM [14], được minh họa trong Hình 20, học cách hiểu tài liệu thông qua sự kết hợp:

- **2D position embeddings:** mô hình mã hóa thông tin vị trí không gian của từ, giúp hệ thống hiểu được bố cục, sự căn lề và khoảng cách tương đối giữa các thành phần văn bản.
- **Image embeddings:** các đặc trưng thị giác từ vùng ảnh chứa văn bản (ROI) được trích xuất bởi Faster R-CNN và kết hợp vào quá trình huấn luyện, giúp mô hình nắm bắt được các dấu hiệu thị giác như phong chữ, kiểu dáng và ngữ cảnh đồ họa.

Sự kết hợp này là cốt lõi để xử lý các câu hỏi đòi hỏi khả năng đọc và suy luận trên các tài liệu giàu thông tin như biểu đồ, hóa đơn hoặc poster quảng cáo.

3. Vấn đề language priors



Hình 21: Minh họa một ví dụ cụ thể của vấn đề language priors. Ảnh trích từ [15].

Một trong những hạn chế lớn của các mô hình VQA hiện nay là xu hướng học dựa trên phân phối dữ liệu thay vì thực sự suy luận trên hình ảnh [15, 16]. Như ví dụ trong Hình 21 minh họa, mô hình sẽ luôn trả lời “màu vàng” cho câu hỏi về màu sắc của trái chuối bất kể ảnh là gì. Hướng giải quyết cho vấn đề này cần tập trung vào việc giảm thiểu sự phụ thuộc trên để tăng tính ổn định cho mô hình:

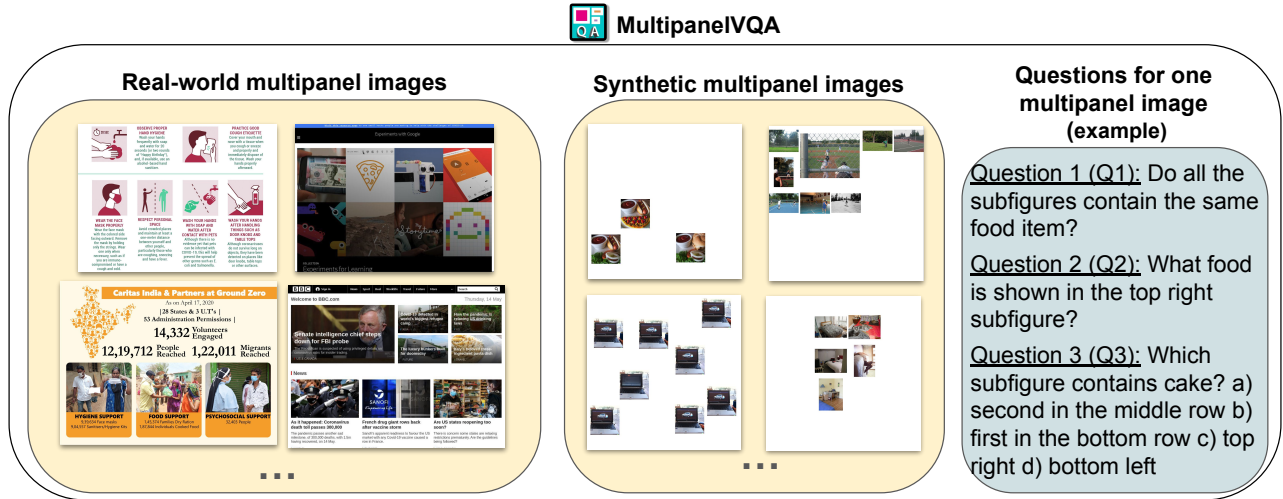
- **Chiến lược xây dựng mô hình:** thiết kế các nhánh phụ để loại bỏ các bias ngôn ngữ trong quá trình huấn luyện.
- **Tăng cường dữ liệu:** sử dụng các phương pháp tạo mẫu đối nghịch hoặc thay đổi ngữ nghĩa câu hỏi để buộc mô hình phải quan sát hình ảnh kỹ hơn trước khi đưa ra dự đoán.

Mục tiêu của hướng đi này không chỉ đơn thuần là tăng độ chính xác mà quan trọng hơn là đảm bảo mô hình thực sự học được khả năng hiểu hình ảnh và suy luận đúng đắn.

4. Các bài toán VQA mới

Bên cạnh các bài toán VQA truyền thống mà ta đã tìm hiểu ở trên, cộng đồng nghiên cứu hiện đang hướng tới các thách thức VQA mới đa dạng và sát với thực tế hơn:

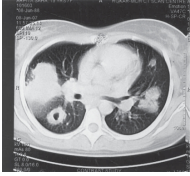
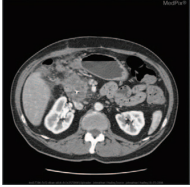
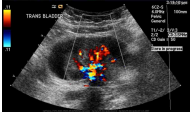
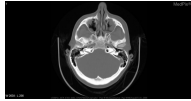
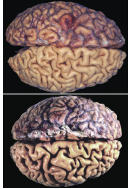
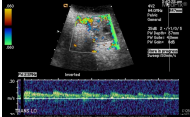
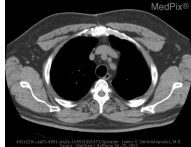
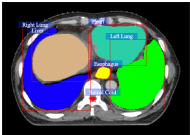
- **Multipanel VQA:** Khác với VQA truyền thống chỉ tập trung vào một bức ảnh đơn lẻ, bài toán này yêu cầu mô hình phải đọc một ảnh có chứa nhiều các hình ảnh con bên trong.



Hình 22: Ví dụ về Multipanel VQA trên dữ liệu thực tế như báo chí, poster và dữ liệu tổng hợp, đòi hỏi khả năng so sánh và tổng hợp thông tin giữa các ảnh con [17].

Như minh họa trong Hình 22, câu hỏi không chỉ nhắm vào một đối tượng cụ thể mà thường yêu cầu sự so sánh hoặc tổng hợp thông tin. Điều này đòi hỏi mô hình phải có khả năng suy luận không gian và nhận biết vị trí giữa các khung hình để đưa ra câu trả lời chính xác.

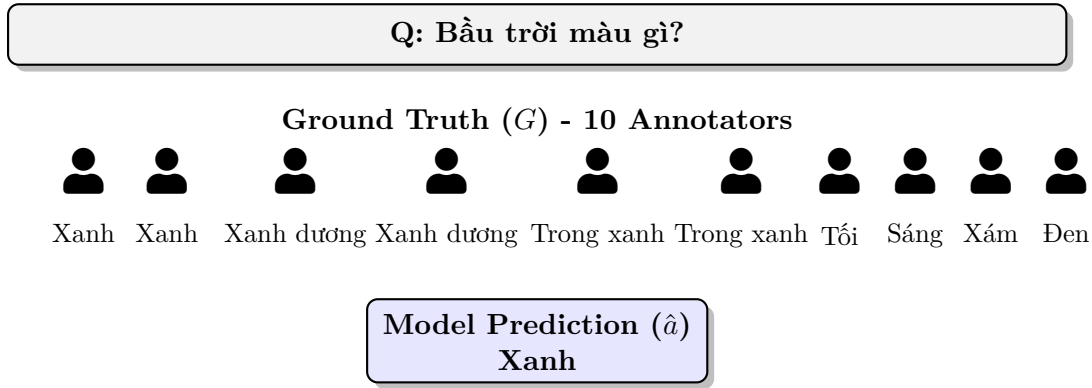
- **Medical VQA:** Đây là hướng ứng dụng chuyên sâu nhằm hỗ trợ chẩn đoán lâm sàng tự động. Thay vì là các bức ảnh tự nhiên, đầu vào của bài toán này là các hình ảnh y tế chuyên ngành như X-quang, MRI, CT hoặc siêu âm.

VQA-Med-2018 [33]		Q: What does the ct scan of thorax show? A: bilateral multiple pulmonary nodules		Q: Is the lesion associated with a mass effect? A: no
VQA-RAD [48]		Organ System Q: What is the organ system? A: Gastrointestinal	Object/Condition Presence Q: Is there gastric fullness? A: yes	Positional Q: What is the location of the mass? A: head of the pancreas
VQA-Med-2019 [14]		Modality Q: what imaging method was used? A: us-d - doppler ultrasound		Plane Q: which plane is the image shown in? A: axial
RadVisDial [46]		Q: Airspace opacity? A: Yes Q: Fracture? A: Not in report	Q: Lung lesion? A: No Q: Pneumonia? A: Yes	
PathVQA [35]		Q: What have been stripped from the bottom half of each specimen to show the surface of the brain? A: meninges		Q: Is remote kidney infarct replaced by a large fibrotic scar? A: yes
VQA-Med-2020 [13]		Q: what abnormality is seen in the image? A: ovarian torsion		Q: what is abnormal in the ct scan? A: partial anomalous pulmonary venous return
SLAKE [57]		Q: Does the image contain left lung? A: Yes	Q: What is the function of the rightmost organ in this picture? A: Breathe	

Hình 23: Tổng hợp một số bộ dữ liệu Medical VQA phổ biến với đa dạng phương thức từ chụp cắt lớp CT đến giải phẫu bệnh Pathology. Ảnh trích từ [18].

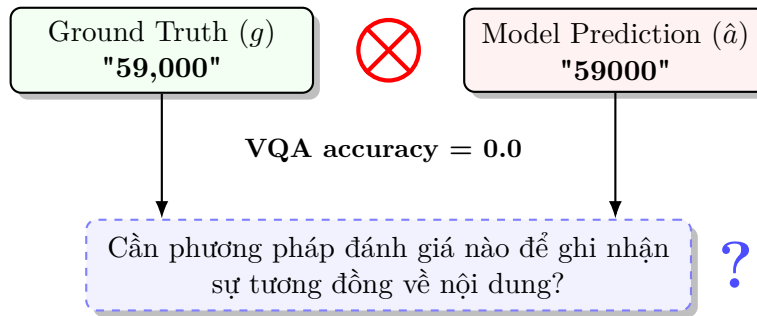
Thách thức chính được thể hiện qua Hình 23 là sự đa dạng về phương thức hình ảnh và độ khó của câu hỏi, từ việc xác định bộ phận cơ thể đến phát hiện bất thường bệnh lý. Mô hình không chỉ cần khả năng thị giác máy tính mà còn phải được tích hợp kiến thức y khoa để đảm bảo độ chính xác cao trong chẩn đoán.

VI. Câu hỏi trắc nghiệm



Hình 24: Phân bố chi tiết 10 người gán nhãn và dự đoán của mô hình.

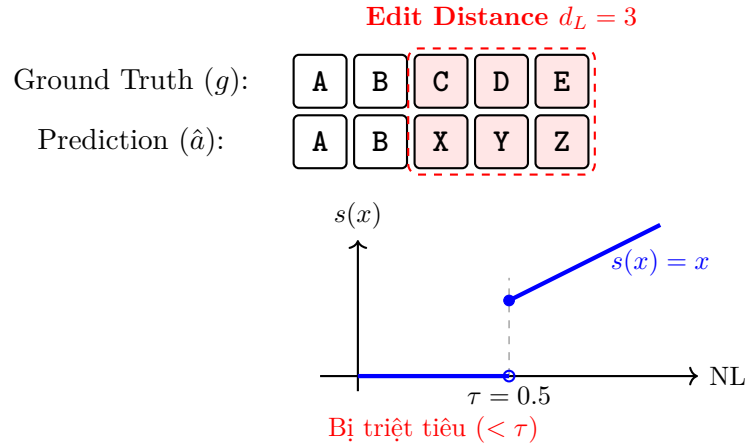
- Dựa vào phân bố câu trả lời của 10 người gán nhãn và dự đoán của mô hình trong Hình 24, hãy tính điểm VQA accuracy chính xác cho trường hợp này.
 - 1.00
 - 0.67
 - 0.20
 - 0.33



Hình 25: Sự hạn chế của độ đo Exact Match khi gặp sai lệch định dạng văn bản.

- Xét một trường hợp câu trả lời của mô hình đúng về mặt nội dung nhưng khác biệt về định dạng như Hình 25, phương pháp hoặc độ đo nào sau đây có thể được sử dụng để đánh giá khách quan hơn VQA accuracy?
 - Chỉ sử dụng precision và recall để thay thế hoàn toàn accuracy.
 - Sử dụng các bước tiền xử lý văn bản như loại bỏ ký tự đặc biệt và dấu câu trước khi so khớp.
 - Chuyển sang sử dụng độ đo ANLS.

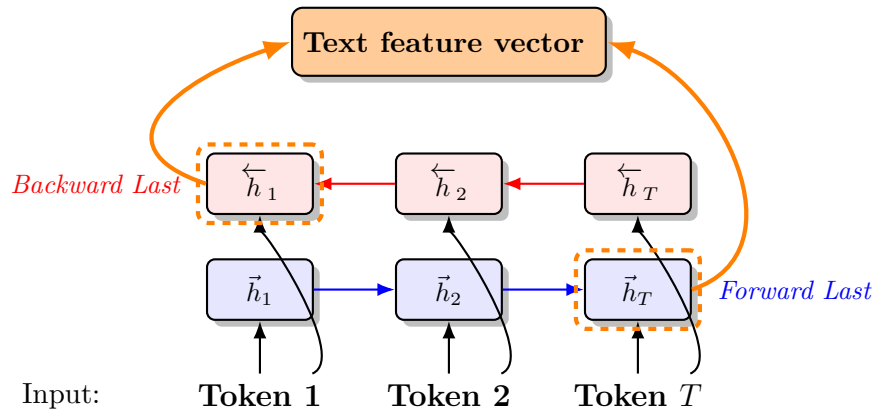
D. Cả B và C đều đúng.



Hình 26: Minh họa khoảng cách chỉnh sửa và cơ chế lọc ngưỡng của ANLS.

3. Xét ví dụ về so khớp hai chuỗi văn bản. Giả sử nhãn đúng là chuỗi g và mô hình dự đoán chuỗi $\hat{a}$ bị sai lệch 3 ký tự, khoảng cách Levenshtein $d_L = 3$, như minh họa trong Hình 26. Điểm số ANLS cuối cùng cho mẫu này là bao nhiêu?

- A. 0.40
- B. 0.60
- C. 0.00
- D. 0.20

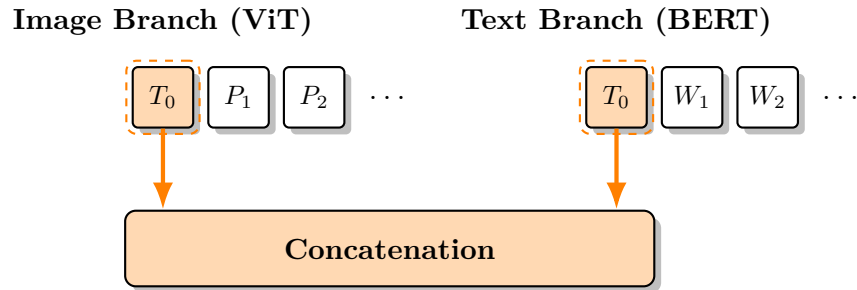


Hình 27: Sơ đồ luồng dữ liệu trích xuất đặc trưng văn bản trong mạng Bi-LSTM.

4. Trong kiến trúc CNN + Bi-LSTM, vector đặc trưng văn bản cuối cùng trước khi đi vào fusion được tạo ra từ thành phần nào của mạng LSTM? Hãy quan sát sơ đồ luồng dữ liệu trong Hình 27.

- A. Lấy hidden state tại bước thời gian cuối cùng của lớp LSTM chiều thuận.

- B. Element-wise addition giữa hidden state của chiều thuận và chiều nghịch.
- C. Nối hidden state cuối cùng của chiều thuận và chiều nghịch.
- D. Lấy trung bình cộng của tất cả các hidden state tại mọi bước thời gian.



Hình 28: Trích xuất trực tiếp đặc trưng từ token đại diện T_0 vào phần kết hợp fusion.

5. Trong phương pháp ViT + BERT, như được minh họa trong Hình 28, thành phần nào được sử dụng làm đại diện đặc trưng cho toàn bộ hình ảnh và câu hỏi trước khi được concat?
 - A. Trung bình cộng của tất cả các patch tokens và word tokens.
 - B. Token [CLS] của ViT cho ảnh và token [SEP] của BERT cho văn bản.
 - C. Token [CLS] của ViT cho ảnh và token [CLS] của BERT cho văn bản.
 - D. Flatten toàn bộ output tensor của ViT và BERT.
6. Hình bên dưới mô tả tổng quát mô hình VQA với đầu vào gồm một hình ảnh kích thước 224 x 224 và một câu hỏi liên quan đến nội dung trong hình ảnh đó. Hai đầu vào đi qua hai thành phần trích xuất đặc trưng phù hợp để nhận được hai vector đặc trưng tương ứng. Hai vector này được tổng hợp bằng cách nối chúng lại với nhau, với vector hình ảnh đứng trước. Đặc trưng được tổng hợp sẽ truyền vào Bottleneck và Classifier để dự đoán câu trả lời. Hỏi đoạn code nào xây dựng chính xác mô hình được minh họa?

A.

```

1 import torch
2 import torch.nn as nn
3
4 class TinyVQA(nn.Module):
5     def __init__(self, hidden_dim, num_answers,
6                 vocab_size=1000, image_size=224):
7         super().__init__()
8         self.flatten = nn.Flatten()
9         self.vision_encoder = nn.Linear(image_size*image_size, hidden_dim) #
10        self.language_encoder = nn.Embedding(vocab_size, hidden_dim)
11        self.classifier = nn.Linear(hidden_dim, num_answers) #
12
13    def forward(self, img, q_ids):
14        x_img = self.flatten(img)
15        v = self.vision_encoder(x_img)
16

```

```

17         x_txt = self.language_encoder(q_ids)
18         t = x_txt.mean(dim=1)
19
20         z = torch.cat([v, t], dim=-1) #
21         out = self.classifier(z)
22         return out

```

B.

```

1 import torch
2 import torch.nn as nn
3
4 class TinyVQA(nn.Module):
5     def __init__(self, hidden_dim, num_answers,
6                 vocab_size=1000, image_size=224):
7         super().__init__()
8         self.flatten = nn.Flatten()
9         self.vision_encoder = nn.Linear(3*image_size*image_size, hidden_dim) #
10        self.language_encoder = nn.Embedding(vocab_size, hidden_dim)
11        self.classifier = nn.Linear(hidden_dim*2, num_answers) #
12
13    def forward(self, img, q_ids):
14        x_img = self.flatten(img)
15        v = self.vision_encoder(x_img)
16
17        x_txt = self.language_encoder(q_ids)
18        t = x_txt.mean(dim=1)
19
20        z = torch.cat([t, v], dim=-1) #
21        out = self.classifier(z)
22        return out

```

C.

```

1 import torch
2 import torch.nn as nn
3
4 class TinyVQA(nn.Module):
5     def __init__(self, hidden_dim, num_answers,
6                 vocab_size=1000, image_size=224):
7         super().__init__()
8         self.flatten = nn.Flatten()
9         self.vision_encoder = nn.Linear(image_size*image_size, hidden_dim) #
10        self.language_encoder = nn.Embedding(vocab_size, hidden_dim)
11        self.classifier = nn.Linear(hidden_dim*2, num_answers) #
12
13    def forward(self, img, q_ids):
14        x_img = self.flatten(img)
15        v = self.vision_encoder(x_img)
16
17        x_txt = self.language_encoder(q_ids)
18        t = x_txt.mean(dim=1)

```

```

19
20     z = torch.cat([t, v], dim=-1)  #
21     out = self.classifier(z)
22     return out

```

D.

```

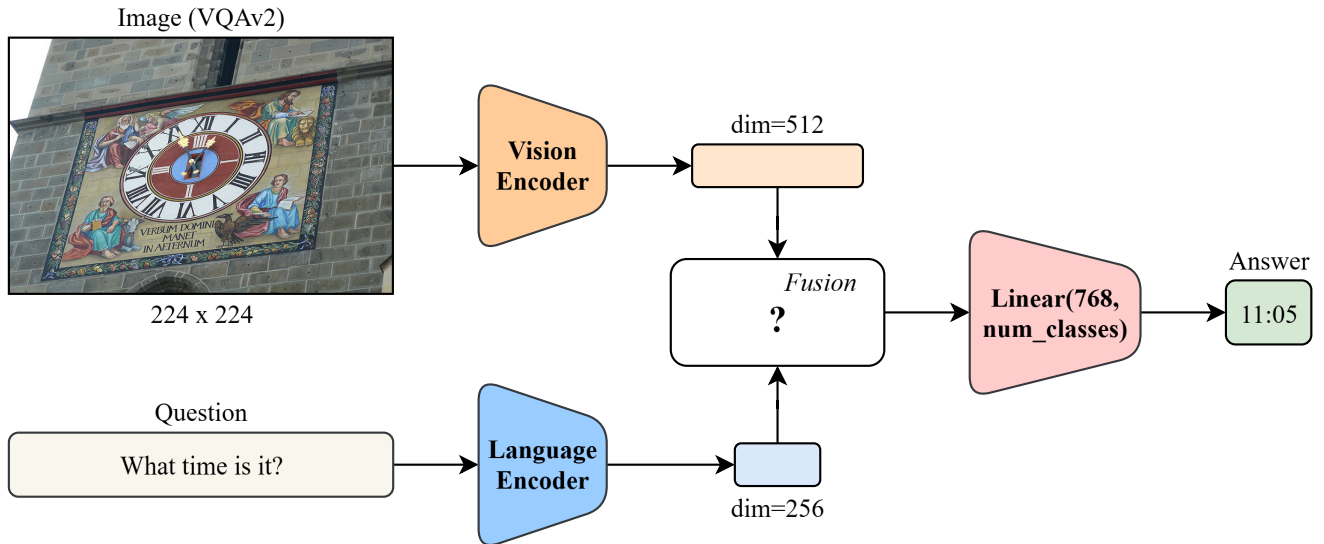
1 import torch
2 import torch.nn as nn
3
4 class TinyVQA(nn.Module):
5     def __init__(self, hidden_dim, num_answers,
6                   vocab_size=1000, image_size=224):
7         super().__init__()
8         self.flatten = nn.Flatten()
9         self.vision_encoder = nn.Linear(3*image_size*image_size, hidden_dim)  #
10        self.language_encoder = nn.Embedding(vocab_size, hidden_dim)
11        self.classifier = nn.Linear(hidden_dim*2, num_answers)  #
12
13    def forward(self, img, q_ids):
14        x_img = self.flatten(img)
15        v = self.vision_encoder(x_img)
16
17        x_txt = self.language_encoder(q_ids)
18        t = x_txt.mean(dim=1)
19
20        z = torch.cat([v, t], dim=-1)  #
21        out = self.classifier(z)
22        return out

```

7. Với mô hình tổng quát tương tự như câu hỏi trên, nhưng trong trường hợp hai thành phần trích xuất đặc trưng trả về hai vector đặc trưng có kích thước khác nhau như hình minh họa bên dưới. Với mạng này, dựa vào các cách kết hợp đặc trưng được đã giới thiệu trong bài, gồm:

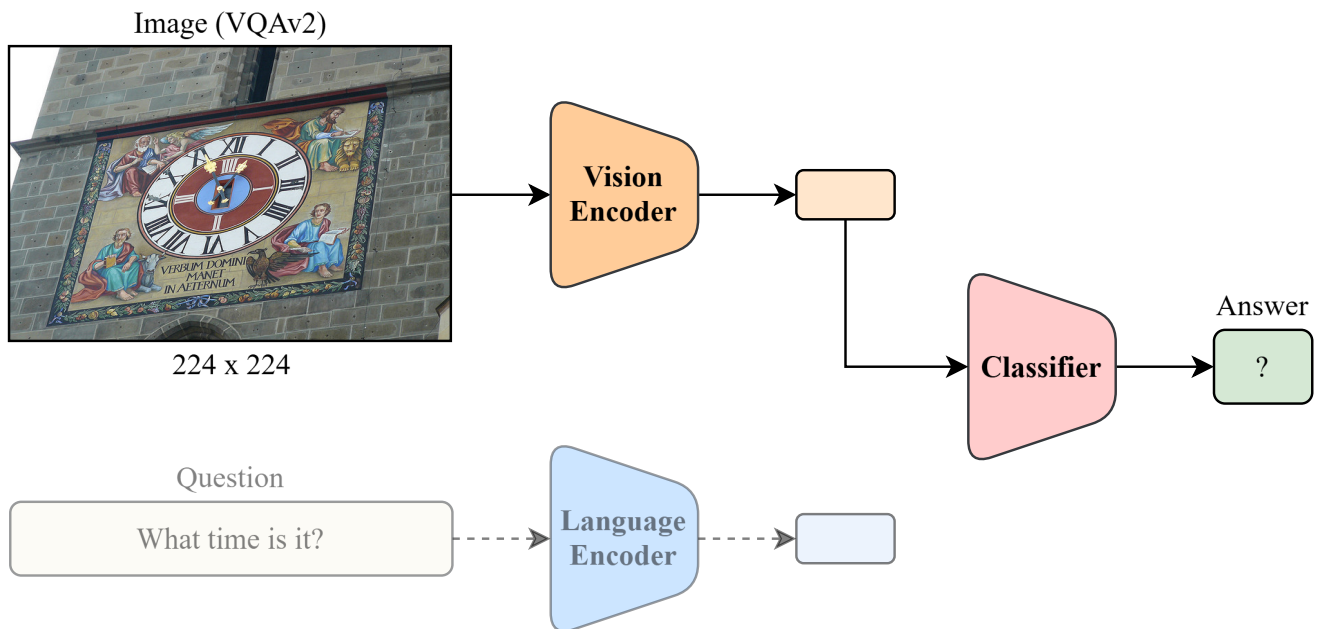
- Nhân hai vector.
- Cộng hai vector.
- Nối hai vector.

Vậy có bao nhiêu cách có thể sử dụng được?



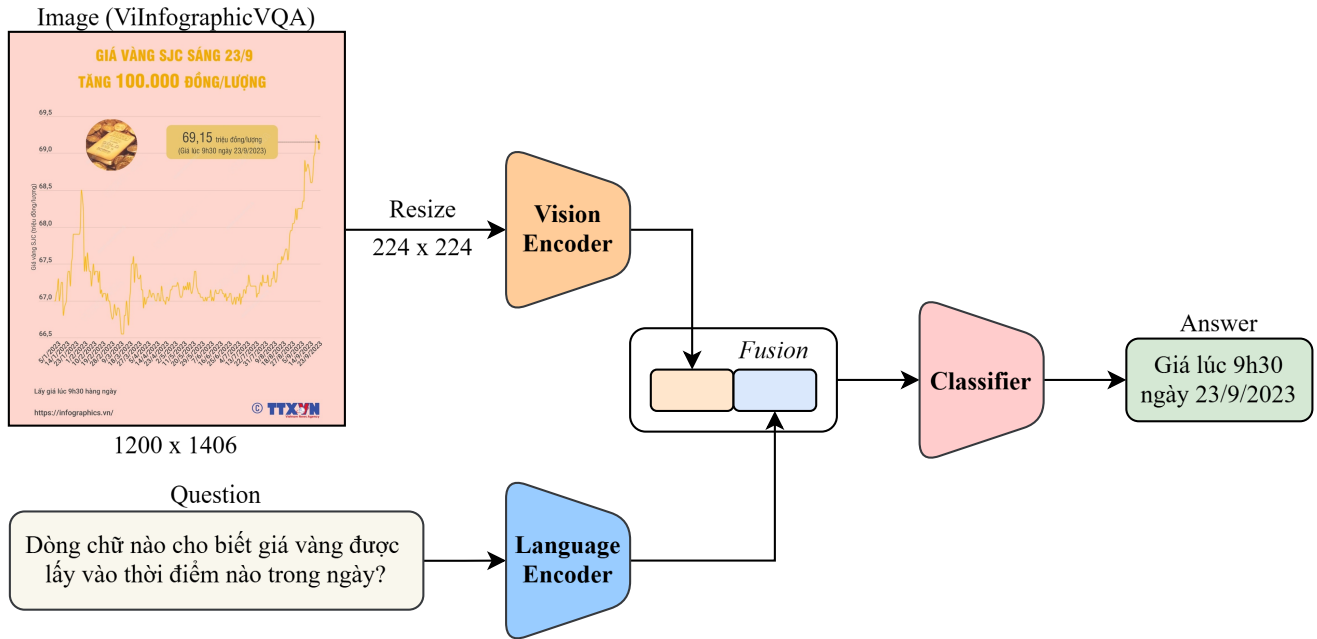
- A. 0.
- B. 1.
- C. 2.
- D. 3.

8. Trong trường hợp mô hình VQA không nhận được thông tin về ngôn ngữ, xem hình minh họa bên dưới, mô hình có thể khả năng trả lời đúng hay không, vì sao?



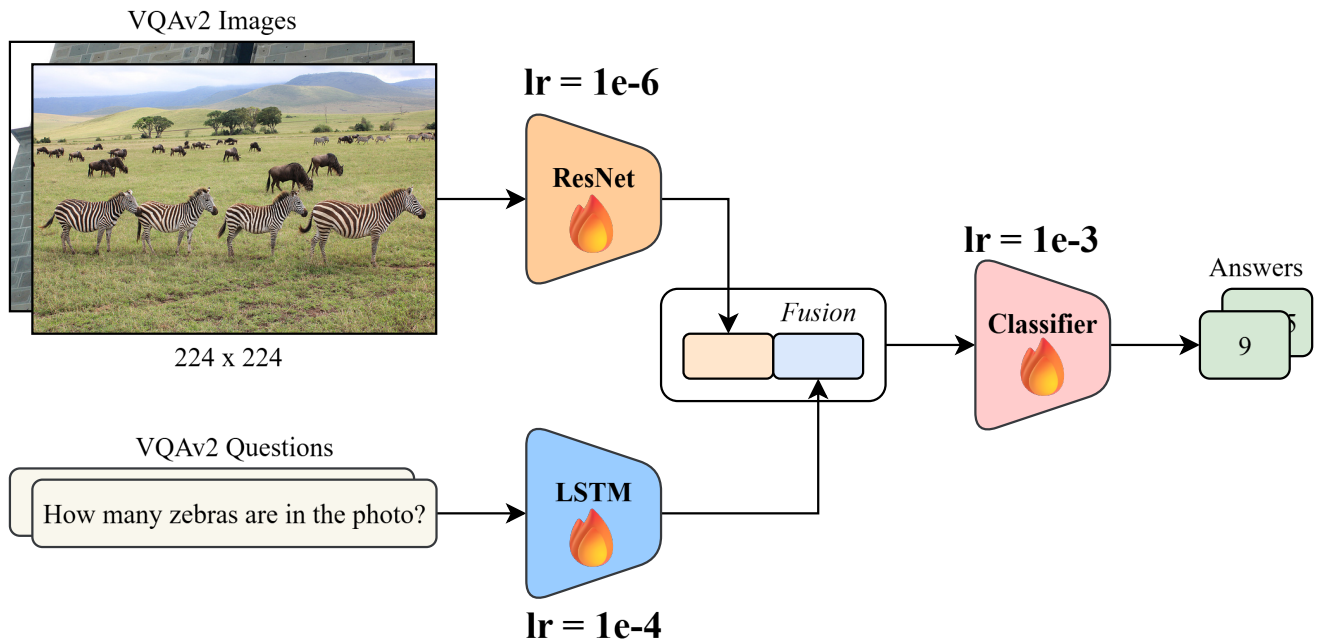
- A. Không thể, vì không có câu hỏi thì mô hình chắc chắn trả lời sai vì có rất nhiều đáp án để lựa chọn.
- B. Có thể, vì chỉ cần nhìn ảnh là đủ thông tin để suy ra đáp án.

- C. Có thể, vì mô hình có thể dự đoán theo thiên lệch của dữ liệu hoặc đáp án hay gặp.
- D. Có thể, vì mô hình có thể dự đoán ngẫu nhiên và vô tình đúng.
9. Với bộ dữ liệu tiếng Việt như ViInfographicVQA là ảnh infographic với nhiều chữ, kích thước lớn như 1200 x 1406, vì sao các mô hình VQA được đề xuất trong bài (với Language Encoder là PhoBERT) thường giảm hiệu suất đáng kể so với bộ dữ liệu tiếng Anh VQAv2, chứa hình ảnh thiên về các hoạt động hằng ngày và kích thước nhỏ 224 x 224?



- A. Vì thành phần trích xuất ngôn ngữ được đề xuất không hiểu tiếng Việt.
- B. Vì resize làm mất thông tin của hình ảnh gốc.
- C. Vì mô hình bị thiên lệch về đặc trưng hình ảnh.
- D. Vì hình ảnh có nhiều chữ nhưng không có mô hình OCR để trích xuất các thông tin này.
10. Với kiến trúc mô hình VQA sử dụng ResNet và LSTM tương tự phương pháp III.2., nhưng khi huấn luyện, ResNet cũng được học thay vì bị đóng băng như trong baseline mẫu, đồng thời các thành phần cũng được điều chỉnh giá trị learning rate khác nhau như trong hình minh họa bên dưới. Theo đó, với cấu hình này, kết quả đánh giá của mô hình này như thế nào? Biết kết quả mô hình baseline ban đầu là:
- Test loss: 2.0255,
 - Test top-1 accuracy: 52.70%,
 - Test top-5 accuracy: 80.20%,
 - Test ANLS: 0.5466.

Lưu ý: Mô hình đã được huấn luyện sẵn, hãy xem qua notebook [này](#), sau đó, tiến hành tải trọng số mô hình và chạy đánh giá trên tập test để ghi nhận kết quả.



- A. Test loss thấp hơn.
- B. Tỷ hơn trên test loss.
- C. Tỷ hơn trên cả test top-1 accuracy và test top-5 accuracy.
- D. Tỷ hơn trên test ANLS.

VII. Tài liệu tham khảo

- [1] Y. Goyal, T. Khot, D. Summers-Stay, et al., “Making the v in vqa matter: Elevating the role of image understanding in visual question answering”, in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 6904–6913.
- [2] T.-T. Van-Dinh, H.-D. Tran, T.-B. Duong, et al., *Vinfographicvqa: A benchmark for single and multi-image visual question answering on vietnamese infographics*, 2025. arXiv: [2512.12424 \[cs.CV\]](#).
- [3] S. Antol, A. Agrawal, J. Lu, et al., “Vqa: Visual question answering”, in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 2425–2433.
- [4] P. Anderson, X. He, C. Buehler, et al., “Bottom-up and top-down attention for image captioning and visual question answering”, in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 6077–6086.
- [5] J. Lu, D. Batra, D. Parikh, et al., “Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks”, *Advances in neural information processing systems*, vol. 32, 2019.
- [6] H. Tan and M. Bansal, “Lxmert: Learning cross-modality encoder representations from transformers”, in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 5100–5111.
- [7] S. Bai, Y. Cai, R. Chen, et al., *Qwen3-vl technical report*, 2025. arXiv: [2511.21631 \[cs.CV\]](#).
- [8] J. Li, D. Li, C. Xiong, et al., “Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation”, in *ICML*, 2022.
- [9] H. Liu, C. Li, Q. Wu, et al., “Visual instruction tuning”, in *NeurIPS*, Published in NeurIPS 2023, often cited as 2024 in recent reviews, 2023.
- [10] W. Wang, Z. Gao, L. Gu, et al., “Internvl3. 5: Advancing open-source multimodal models in versatility, reasoning, and efficiency”, *arXiv preprint arXiv:2508.18265*, 2025.
- [11] A. F. Biten, R. Tito, A. Mafla, et al., “Scene text visual question answering”, in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 4291–4301.
- [12] J.-H. Kim, J. Jun, and B.-T. Zhang, “Bilinear attention networks”, *Advances in neural information processing systems*, vol. 31, 2018.
- [13] Z. Yu, J. Yu, Y. Cui, et al., “Deep modular co-attention networks for visual question answering”, in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 6281–6290.
- [14] Y. Xu, M. Li, L. Cui, et al., “Layoutlm: Pre-training of text and layout for document image understanding”, in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 1192–1200.
- [15] R. Cadene, C. Dancette, M. Cord, et al., “Rubi: Reducing unimodal biases for visual question answering”, *Advances in neural information processing systems*, vol. 32, 2019.

- [16] A. Vo, K.-N. Nguyen, M. R. Taesiri, et al., “Vision language models are biased”, in *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026.
- [17] Y. Fan, J. Gu, K. Zhou, et al., “Muffin or chihuahua? challenging multimodal large language models with multipanel vqa”, in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 6845–6863.
- [18] Z. Lin, D. Zhang, Q. Tao, et al., “Medical visual question answering: A survey”, *Artificial Intelligence in Medicine*, vol. 143, p. 102611, 2023.

Bảng thuật ngữ

Thuật ngữ	Tạm dịch	Mô tả
<i>ANLS</i>	Độ tương đồng Levenshtein chuẩn hóa	Average normalized Levenshtein similarity, độ đo dựa trên khoảng cách chỉnh sửa chuỗi ký tự. ANLS thường được dùng trong các bài toán TextVQA để đánh giá mức độ khớp của câu trả lời một cách mềm dẻo, chấp nhận các sai số nhỏ do lỗi OCR
<i>Backbone</i>	Mạng xương sống	Một mạng nơ-ron nền tảng thường đã được huấn luyện trước trên tập dữ liệu lớn, đóng vai trò trích xuất đặc trưng cơ bản từ dữ liệu đầu vào
<i>BERT</i>	Mô hình BERT	Bidirectional encoder representations from Transformers, một mô hình ngôn ngữ được huấn luyện trước dựa trên kiến trúc Transformer để học biểu diễn ngữ cảnh hai chiều của từ
<i>Bi-directional LSTM (BiLSTM)</i>	LSTM hai chiều	Một biến thể của LSTM bao gồm hai lớp mạng chạy song song theo hai hướng ngược nhau, cho phép mô hình nắm bắt ngữ cảnh toàn diện từ hai chiều của chuỗi dữ liệu
<i>Concatenation</i>	Phép nối	Phép toán kết hợp hai hoặc nhiều vector lại với nhau theo một chiều nhất định để tạo thành một vector dài hơn chứa thông tin của tất cả các thành phần
<i>Convolutional neural network (CNN)</i>	Mạng nơ-ron tích chập	Loại mạng nơ-ron chuyên biệt cho việc xử lý dữ liệu có cấu trúc lưới như hình ảnh, sử dụng các phép tích chập để trích xuất đặc trưng không gian
<i>Flattening</i>	Làm phẳng	Kỹ thuật biến đổi một tensor đa chiều thành một vector một chiều, thường được sử dụng để chuyển đổi đặc trưng ảnh trước khi đưa vào các lớp kết nối đầy đủ

Thuật ngữ	Tạm dịch	Mô tả
<i>Long short-term memory (LSTM)</i>	Bộ nhớ ngắn-dài hạn	Một biến thể của mạng nơ-ron hồi quy có khả năng học các phụ thuộc xa trong dữ liệu chuỗi nhờ cơ chế cổng kiểm soát luồng thông tin
<i>Multi-layer perceptron (MLP)</i>	Mạng nơ-ron đa lớp	Mạng nơ-ron truyền thẳng bao gồm ít nhất ba lớp nút là lớp đầu vào, lớp ẩn và lớp đầu ra, sử dụng hàm kích hoạt phi tuyến để học các mối quan hệ phức tạp
<i>Optical character recognition (OCR)</i>	Nhận dạng ký tự quang học	Công nghệ cho phép chuyển đổi hình ảnh chứa văn bản đánh máy hoặc viết tay thành dữ liệu văn bản máy tính có thể chỉnh sửa và tìm kiếm được
<i>TF-IDF</i>	Trọng số tần suất nghịch đảo	Term frequency-inverse document frequency, kỹ thuật thống kê dùng để đánh giá tầm quan trọng của một từ trong văn bản dựa trên tần suất xuất hiện của nó
<i>Top-5 Accuracy</i>	Độ chính xác Top-5	Chỉ số đánh giá xem câu trả lời đúng có nằm trong danh sách 5 dự đoán có xác suất cao nhất của mô hình hay không, thường dùng để kiểm tra khả năng bao quát của mô hình
<i>Vision Transformer (ViT)</i>	Mô hình Transformer thị giác	Kiến trúc áp dụng cơ chế self-attention trực tiếp lên các mảnh ảnh, xử lý hình ảnh như một chuỗi các token thay vì sử dụng phép tích chập truyền thống
<i>VQA Accuracy</i>	Độ chính xác VQA	Một trong các thước đo tiêu chuẩn để đánh giá mô hình VQA, so sánh câu trả lời dự đoán với các nhãn thực tế. Điểm số thường được tính dựa trên quy tắc đồng thuận, cần ít nhất 3/10 người gán nhãn đồng ý để đạt điểm tối đa
<i>Word Embedding</i>	Nhúng từ	Kỹ thuật biểu diễn các từ vựng trong ngôn ngữ tự nhiên thành các vector số học trong không gian liên tục sao cho giữ được mối quan hệ ngữ nghĩa giữa chúng

Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại [đây](#).
2. **Demo:** Chương trình demo cho nội dung trong bài có thể được truy cập tại [đây](#).
3. **Hint:** Các file code gợi ý có thể được tải tại [đây](#).
4. **Solution:** Các file code cài đặt thực nghiệm có thể được tải tại [đây](#).
5. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,6K thành viên



Hình 29: Hình ảnh group facebook AIO Q&A.